

ARQUITECTURA AGÉNTICA ANTIGRAVITY — Auditoría Forense 360°

Multiagente + Sistema Completo

Fecha: 2026-08-08, re-verificado y ampliado 2026-08-10 (\$0-D) y 2026-08-11/12 (\$0-E...\$0-K en el camino, \$0-L Migración A confirmada en Supabase con evidencia)

Auditor: Chief AI Architect / Auditor Forense de Sistemas Multiagente / DevSecOps Lead / Chief Software Auditor / System Architect

Alcance: proyecto raíz `c:\2026 AI EGI0C5\Antigravity JS`, **ambas ramas remotas** (`origin/master` y `origin/main`, ver \$0-D). `proyectos/` queda fuera del árbol de trabajo local (repos git independientes, `.gitignore:19-24`) — pero ver \$0-D sobre su relación real con `origin/main`.

Regla de evidencia: cero suposiciones — cada hallazgo cita archivo real. Donde el volumen hizo impracticable la lectura línea-por-línea de decenas de archivos (los skills de `agents/001_ORQUESTADOR_MAESTRO/_archivo_historico/skills_radar_legacy/`, o los 171 commits de `origin/main`), se declara el muestreo usado.

Estado del commit (rama `master`): local `12886c9`, **8 commits adelante de `origin/master`** (`d9e520a`) — sin push, decisión pendiente del usuario. Incluye el trabajo de refactor de `001_ORQUESTADOR_MAESTRO / architecture-gate.cjs` de esta sesión (ruteo estático, motor de resiliencia, audit trail JSONL).

Estado de la rama `main`: `origin/main` (rama default real del repo en GitHub — `remotes/origin/HEAD -> origin/main`), HEAD en `9dfb577`, último commit **2026-08-10** (hoy). Nunca tocada por esta sesión ni por ninguna versión anterior de este documento.

0. QUÉ CAMBIÓ DESDE LA ÚLTIMA VERSIÓN DE ESTE DOCUMENTO

La versión anterior de `docs/ARQUITECTURA_AGENTICA_ANTIGRAVITY.md` (2026-08-08, madrugada) documentó 7 de 9 scripts sueltos en `agents/` como rotos/fósiles/huérfanos. Desde entonces, en la misma jornada, se ejecutó una remediación real:

Ítem del documento anterior	Estado ahora
<code>agents/auditor-integridad.cjs</code> (fósil, 11/11 rutas inexistentes)	Eliminado
<code>agents/bridge-server.cjs</code> (2º servidor Express, puerto 3001, API rota)	Eliminado
<code>agents/skill-dispatcher.cjs</code> (<code>schema available_skills</code> inexistente)	Eliminado
<code>agents/index.js</code> (<code>import</code> roto a <code>config.js</code>)	Eliminado
<code>agents/ContextManager.js</code> (referencia a proyecto purgado)	Eliminado
<code>agents/Agente001/050/051/052.js</code> (huérfanos, solo importados por <code>index.js</code>)	Eliminados
<code>MiniMaxChat.jsx</code> + <code>/api/openrouter/*</code>	Eliminados por completo — decisión de producto: el backend solo expone Claude
<code>CLAUDE_MODEL</code> hardcodedo en 2 archivos distintos	Centralizado vía <code>PRIMARY_AI_MODEL</code> en <code>.env</code> (<code>claude-sonnet-4-6</code>)
<code>claves_privadas.txt</code> (26 líneas, incluía JWT legacy <code>service_role</code> de Supabase, token de gestión <code>sbp...</code> , Hostinger, GitLab con password en texto plano)	Reducido a 7 líneas — solo lo que coincide con <code>.env</code> activo (backup fuera del repo)
Historial git local/remoto sin ancestro común	Resuelto — <code>origin/master</code> ahora = <code>d9e520a</code> (ver \$6.4 para el detalle de riesgo que esto implicó)
<code>GET /api/health</code> — falso positivo (solo verificaba env var)	Corregido — ping real cacheado 120s

Lo que **no** cambió y sigue vigente tal cual: los 25 skills legacy (ahora en `agents/001_ORQUESTADOR_MAESTRO/_archivo_historico/skills_radar_legacy/`, huérfanos, sin consumidor), la brecha `IDENTITY.md-vs-ejecución` en 052/056, la ausencia de panel `/admin`, y los 4 sistemas de agentes coexistentes (A/B/C/E).

0-B. SEGUNDA RONDA (2026-08-08, misma jornada) — reparación de un renombramiento a medias iniciado por Gemini

Otra sesión (Gemini, según reportó el usuario) comenzó a reenumerar el Escuadrón Élite de 5 a 8 roles y se cortó a mitad de camino por límite de tokens, dejando 2 archivos modificados sin commitear y en contradicción directa entre sí: `AGENTS.md` (nueva topología, 8 roles, refería al rol

nuevo como `008_AUDITOR_DE_CODIGO`) y `.claude/agents/architect.md` (mismo rol, pero como `006_AUDITOR_DE_CODIGO` , dos veces). Detectado con `git status / git diff` en frío antes de tocar nada — ninguna carpeta física de `agents/` había sido tocada todavía.

El usuario eligió completar el esquema de Gemini tal cual (001-008), asumiendo `008` como el número correcto. Esto exigía reenumerar carpetas reales que colisionaban con los nuevos IDs de rol — un paso que Gemini nunca llegó a proponer ni ejecutar.

Trabajo completado esta ronda:

- 5 carpetas renombradas (`git mv` , historial preservado): `000_ORQUESTADOR` → `001_ORQUESTADOR_MAESTRO` , `001_gestor_datos` → `009_gestor_datos` , `002_redactor_tecnico` → `010_redactor_tecnico` , `005_Radar1_minero` → `011_Radar1_minero` , `006_Radar2_Estratega` → `012_Radar2_Estratega` .
- `ESCUADRON_ELITE` en `architecture-gate.cjs` reescrito completo: 8 roles, subordinados reales reasignados por función (ej. los 7 subordinados del antiguo `002_INGENIERIA_TOTAL` pasan a `005_INGENIERO_BACKEND` , ninguno calificaba como frontend).
- Contradicción 006/008 resuelta en `architect.md` (ganó 008, coincide con `AGENTS.md`).
- **Hallazgo adicional durante la reparación, no relacionado con Gemini:** `skills/ag_skills_registry.json` ya tenía 25 rutas rotas desde el archivado de la ronda anterior de hoy (nunca se actualizó ese registro al mover los skills a `_archivo_historico/`) — reparado en la misma pasada, con las skills archivadas ahora marcadas explícitamente como tales (`skills_archivadas_2026-08-08` , separadas del único skill real que vivía en la misma lista, `Skill_Protocolo_Fuente_Unica`).
- `sync_registry.cjs` ahora excluye `_archivo_historico/` de su escaneo — sin esto, redescubriría las 25 skills archivadas como "nuevas" en cada corrida.
- `ANTHROPIC_MODEL` hardcodeado por tercera vez en `architecture-gate.cjs` (se había centralizado en `server.js` y `m1Pipeline.js` , se pasó por alto este) — corregido a `process.env.PRIMARY_AI_MODEL` .
- `IDENTITY.md` de 050/052/056 y del propio `001_ORQUESTADOR_MAESTRO` actualizados (referencias cruzadas a las carpetas renombradas).
- `AGENTS.md` §IV-B corregida: los agentes `06X` que Gemini clasificó como "Agentes de Apoyo" del Escuadrón Élite en realidad viven en `.agent/agents/` (Sistema B, scaffold genérico de terceros) — no son parte de este sistema en absoluto.

0-C. TERCERA RONDA (2026-08-08, misma jornada) — auditoría de calidad real de los 12 skills activos + de los 8 roles del Escuadrón Élite en sí mismos

Pedido explícito del usuario: no solo confirmar que los skills existen (§0/§0-B ya lo hizo), sino leer su contenido y juzgar si están a la altura de un "Escuadrón Élite". Se leyeron íntegros los 12 archivos de skill activos (no archivados) más los 3 scripts Python de `011_Radar1_minero` . Resultado en detalle en §2.3 (skills) y §1.4 (los 8 roles como entidades, sin sus subordinados).

Bugs propios encontrados y corregidos durante esta lectura (no relacionados con Gemini, míos de la ronda anterior):

- `agents/011_Radar1_minero/radar_oficial.py:8-9` seguía escribiendo a `agents/005_Radar1_minero/...` (nombre viejo) — el barrido de referencias de §0-B solo cubrió `.js/.cjs/.json/.md` , nunca `.py` .
- `agents/001_ORQUESTADOR_MAESTRO/puente_ejecutor.py:10-11` — mismo patrón, seguía apuntando a `agents/000_ORQUESTADOR/...` .
- `agents/001_ORQUESTADOR_MAESTRO/puente_ejecutor.py:17` — `ALLOWED_SCRIPTS` (allowlist de seguridad del daemon) seguía aceptando `000_Orquestador.cjs` , el nombre retirado en §0-B — corregido a `architecture-gate.cjs` . Sin este fix, el daemon habría rechazado el script legítimo mientras técnicamente seguía "permitiendo" uno que ya no existe.

Hallazgo nuevo — "Protocolo Titán" (rol `008_AUDITOR_DE_CODIGO`) es un término huérfano: buscado en todo el proyecto (`*.md` , `*.cjs` , `*.js`) — aparece exactamente 2 veces, ambas la misma frase de una línea (`AGENTS.md` y su espejo en `architecture-gate.cjs`). No hay ningún documento, skill o lógica que lo defina. Es un nombre sin contenido detrás, acuñado por Gemini.

0-D. CUARTA RONDA (2026-08-10) — hallazgo crítico: `origin/master` y `origin/main` son dos historiales sin ancestro común

Pedido explícito del usuario: re-verificar este documento contra el estado real de disco antes de darlo por vigente, no reescribirlo desde cero. Verificación de rutina (`git fetch + git branch -a`) reveló algo que ninguna versión anterior de este documento sabía.

El hallazgo, verificado con comandos de solo lectura, sin tocar ningún archivo:

```
git merge-base origin/master origin/main → sin salida, exit 1 (sin ancestro común)
remotes/origin/HEAD -> origin/main → main es la rama default real en GitHub
origin/main: 171 commits, último 2026-08-10 17:48 (HOY)
origin/master: HEAD local 8 commits adelante, último commit propio ajeno a main
```

Es la misma clase de hallazgo que §6.4 ya documentó una vez (historial local/remoto sin ancestro común, resuelto por un force-push externo) — pero esta vez entre **dos ramas remotas del mismo repo**, nunca detectado hasta ahora porque ninguna auditoría anterior corrió `git fetch`

con visibilidad de todas las ramas antes de concluir.

0-D.1 origin/main no es una variante de Antigravity JS — es otro proyecto

Leyendo origin/main:AGENTS.md y origin/main:.claude/agents/architect.md (vía git show, sin checkout) queda explícito: el proyecto se autodenomina "RadarFondos 360", no Antigravity JS. Cita textual de su propio architect.md:

> "Eres el Agente Arquitecto de RadarFondos 360 [...] Adaptado del patrón ya construido y verificado en Antigravity JS/.claude/agents/architect.md (proyecto raíz) — mismo mandato, criterios ajustados a la arquitectura real de este repo."

Es decir: quien construyó esto ya sabía y documentó explícitamente que Antigravity JS y RadarFondos 360 son proyectos distintos — y aun así ambos terminaron pusheados al mismo repositorio de GitHub (jaansave-CKN/Antigravity-JS), en ramas distintas. Esto coincide con memoria ya registrada del usuario: RadarFondos 360 (proyectos/Proy_03_RadarFondos/ en disco local) comparte la misma base Supabase que el proyecto raíz — no son independientes a nivel de datos, y ahora tampoco lo son a nivel de repositorio git.

Riesgo real, no solo de organización: origin/main:AGENTS.md documenta una Capa 2 de fallback a Supabase REST con service_role (bypasea RLS) para RadarFondos 360. El proyecto raíz (rama master) tiene su propio hallazgo crítico sin resolver de un JWT service_role legacy de Supabase (§6.4, §11). Si ambos proyectos comparten instancia de Supabase (memoria del usuario lo confirma), **ambos riesgos podrían ser la misma superficie de ataque**, no dos hallazgos independientes — pendiente de que el usuario confirme si es la misma instancia/proyecto Supabase antes de revocar credenciales de un lado sin considerar el otro.

0-D.2 Huella agéntica real de origin/main — mucho más chica que la de master, y con su propia deuda de documentación

Auditando agents/, .claude/, .agents/, backend/agents/ y backend/services/ de origin/main (vía git ls-tree / git show, sin checkout — mismo rigor de evidencia que el resto del documento):

Ruta en origin/main	Contenido real	Veredicto
agents/	2 archivos: scraper_core.py + su .pyc cacheado	🟡 Un scraper suelto, no una jerarquía de agentes — nada que ver con el Escuadrón Élite de master
.claude/agents/architect.md	Gate real, adaptado del patrón de master (mismo mandato: solo lectura, bloquea antes de escribir código)	🟢 Genuino — describe con precisión verificable el backend real (server.js ~4800 líneas, backend/routes/, backend/config/database.config.js de 2 capas)
.claude/settings.json	Config de Claude Code del repo	Sin auditar en profundidad — fuera del foco multiagente
.agents/skills/obsidian-context.md	1 skill de contexto Obsidian	🟡 Menor, sin relación con orquestación
backend/agents/	arbolObjetivosAgent.js, normativoAgent.js — 2 agentes de dominio reales	🟢 Pequeño pero con nombres de dominio específicos (no genéricos como los stubs de 050 - 056 en master)
backend/services/geminiCircuitBreaker.js, aiTokenLogger.js	Circuit breaker de cuota + logger de tokens para FinOps — ambos existen de verdad , confirmado por contenido	🟢 Esto es exactamente lo que la auditoría de master marcó como 🔴 AUSENTE (§7.2) — main sí lo construyó

Hallazgo de higiene documental, mismo patrón que "Protocolo Titán" (§0-C) pero en la otra rama:

origin/main:.claude/agents/architect.md cita docs/AUDITORIA_MULTIAGENTE_2026-08-04.md como la auditoría que originó la regla "cero código sin diseño aprobado" — ese archivo no existe en origin/main (verificado, git cat-file -t falla, y no aparece bajo ningún nombre similar en docs/). Es una referencia rota a un documento que o se perdió, o nunca se commiteó, o vivía en otra rama/repo — el mismo tipo de "injerto mal ejecutado" que pidió detectar el prompt original, encontrado ahora en la rama que no se había auditado nunca.

0-D.3 Qué significa esto para el resto de este documento

0-F. SEXTA RONDA (2026-08-11, misma jornada) — 002 resuelve el bloqueo escalado por 005, ADR-0001

El usuario escaló formalmente el bloqueo que 005_INGENIERO_BACKEND reportó al ser creado (§0-E) al rol 002_ARQUITECTO_DE_SOFTWARE, pidiendo veredicto sobre (1) vía de autenticación definitiva y (2) plano de portabilidad de WORM/OCC desde Proy_05_SIG y Proy_03_RadarFondos.

Se leyeron íntegros los 2 archivos de migración candidatos a portar más database.config.js y 010_rls_complete_audit.sql / 005_rls_saas_hardening.sql de Proy_03_RadarFondos (definición real de las funciones RLS que usan). **Hallazgo que cambió el veredicto:** la política RLS del proyecto hermano no es una alternativa gratuita a configurar Third-Party Auth — su rama

`current_tenant_uuid()` exige un GUC de sesión (`app.tenant_id`) que solo funciona con conexión `pg.Pool1` persistente (este proyecto retiró `pg` deliberadamente), y su rama `current_auth_uid()` / `auth.uid()` tiene la *misma* dependencia de Third-Party Auth que bloquea al proyecto raíz. Además, `Proy_03_RadarFondos/backend/config/database.config.js` confirma que su propia Capa 1 (RLS real vía `pg.Pool1`) depende de que el usuario "habilite el pooler en Supabase dashboard" — no garantizado —, y degrada a la misma Capa 2 (REST + `SERVICE_KEY`, bypass total) que el hallazgo original de `005`. Es decir: el proyecto que se iba a copiar como "más seguro" tiene el mismo problema sin resolver, solo que fraseado distinto.

Veredicto emitido, documentado en `docs/ADR/ADR-0001-auth-rls-worm-occ.md`: Third-Party Auth como vía definitiva (no reintroducir `pg`), con el guardrail Node actual (`assertValidTenant` + `WHERE tenant_id` explícito) como control primario mientras tanto — no removerlo. Portabilidad de WORM/OCC partida en 2 migraciones: **Migración A** (triggers append-only + OCC de aplicación) autorizada ya, sin nuevo veredicto; **Migración B** (RLS real sobre esas tablas) bloqueada hasta que el usuario confirme Third-Party Auth activo en el dashboard. `.claude/agents/005-ingeniero-backend.md` actualizado con un bloque "Gate resuelto" que resume estas condiciones.

Hallazgo cruzado no pedido, relevante para §6.4: `database.config.js` de `Proy_03_RadarFondos` documenta en su propio comentario que una credencial `service_role` real quedó commiteada ahí antes y fue/debe ser rotada — dado que ambos proyectos comparten instancia Supabase (memoria ya registrada del usuario), **pendiente que el humano confirme si es la misma credencial ya marcada crítica en §6.4** antes de dar esa revocación por completamente cerrada.

0-G. SÉPTIMA RONDA (2026-08-11, misma jornada) — `005` ejecuta Migración A de ADR-0001

El usuario autorizó explícitamente a `005_INGENIERO_BACKEND` a ejecutar Migración A (WORM + OCC), con Migración B (RLS real) explícitamente fuera de alcance. Entregables reales, no solo documentados:

- `src/modules/formulador/migrations/007_worm_occ_shadow_ledger.sql` — tablas `project_version_hashes` (Fase 4.1) y `security_violations_ledger` (Fase 4.2, Shadow Ledger), ambas append-only vía trigger (`RAISE EXCEPTION` en `UPDATE / DELETE`), con RLS habilitado + política `USING(true)` marcada explícitamente como provisional (no confundir con protección real — Migración B la reemplaza). RPCs: `registrar_version_hash`, `obtener_ultimo_hash`, `registrar_violacion_seguridad`.
- `src/modules/formulador/occGuard.js` — módulo Node nuevo: hash SHA-256 estable (orden de claves normalizado), `assertVersionOrConflict()` (aborta 409 si el hash del cliente no coincide con el último registrado), `registrarViolacionSeguridad()` (best-effort, fire-and-forget, mismo patrón que `aiTokenLogger.js` de `Proy_03_RadarFondos`).
- `src/modules/formulador/FormuladorPgController.js` (`guardarModulo10`) — único endpoint de este controller que reemplaza un recurso existente, por eso es donde se ancla OCC: valida versión antes de escribir, registra el nuevo hash tras escribir (best-effort), y alimenta el Shadow Ledger cuando la RPC rechaza la escritura por no pertenecer al tenant (`tenant_mismatch`) — consumidor real del Shadow Ledger desde el día uno, no una tabla huérfana.
- `src/shared/infrastructure/validation.js` — `schemas.modulo10` acepta `version_hash` opcional (64 chars, sha256 hex).
- Preservado sin alterar, por instrucción explícita:** `assertValidTenant()` y el filtro `WHERE tenant_id` de las RPC existentes — siguen siendo el control primario hasta que Migración B esté activa.
- Respetado el veto duro del ADR:** ningún archivo nuevo reintroduce `pg` ni `SET LOCAL app.tenant_id` — las 2 tablas nuevas se escriben solo vía RPC `SECURITY INVOKER` autocontenida, mismo patrón que el resto del esquema.

Validación de gate ejecutada, resultado real (no maquillado): `node agents/architecture-gate.cjs --check-gate` (modo gratuito, sin costo de API) →  Sin aprobación vigente: el estado de `agents/src/` cambió después de la firma. Esto es el comportamiento **correcto y esperado** del gate — cualquier cambio de código invalida la firma anterior por diseño (`hashEstado()`, autoinvalidante). Aprobar una firma nueva requiere `--aprobar-diseno`, que sí gasta la API de Anthropic — no se ejecutó unilateralmente; queda como acción pendiente a decisión del usuario, no como un fallo de esta ronda. Los 3 archivos JS nuevos/modificados pasaron `node --check` (sintaxis válida) antes de este intento de gate.

0-H. OCTAVA RONDA (2026-08-11, misma jornada) — purga física de `.agent/`, reparación de enlaces rotos

Verificado en disco antes de actuar (no se aceptó la directiva como hecho sin comprobarlo, per protocolo de este proyecto): `.agent/` **efectivamente fue eliminada** (`ls .agent` → "No such file or directory"), y `.claude/agents/architect.md` fue renombrado a `.claude/agents/002-arquitecto-de-software.md` (`git status` mostró el rename R). `git status` también reveló que un actor externo a esta sesión ya había agregado `001-orquestador-maestro.md` y `008-auditor-de-codigo.md` a `.claude/agents/` — no creados por esta sesión, releídos íntegros antes de tocarlos (mismo criterio de "no confiar en cambios ajenos sin re-verificar" documentado en `proyectos/Proy_03_RadarFondos/CLAUDE.md`).

Hallazgo no pedido, pero real: esos 2 archivos nuevos traían sus propios defectos, no solo los enlaces rotos por la purga:

- `001-orquestador-maestro.md` §2 seguía ruteando a `004_INGENIERO_FRONTEND` (nombre retirado 2026-08-11, reemplazado por `004_SENTINELA_FRONTEND` en rondas anteriores de esta misma jornada) — corregido.

- `001-orquestador-maestro.md` §4 reintroducía la clasificación "06X = Agentes de Apoyo" que la ronda §0-B ya había corregido una vez (esos archivos vivían en `.agent/agents/`, recién destruida) — ahora era una referencia a una carpeta que ya no existe en absoluto. Reescrito como nota de fuente única de verdad.
- `008-auditor-de-codigo.md` tenía `name: 006_auditor_de_codigo` en el frontmatter pese a llamarse `008-auditor-de-codigo.md` — el mismo tipo exacto de contradicción 006-vs-008 que §0-C ya había resuelto una vez y bautizado "Protocolo Titán huérfano". Corregido a `008-auditor-de-codigo` (y las 2 menciones del cuerpo del archivo).

Enlaces reparados (la orden original):

- `agents/architecture-gate.cjs` — 11 referencias a `.claude/agents/architect.md` (incluida `ARCHITECT_PROMPT_PATH`, la ruta que de verdad carga el prompt del gate) actualizadas a `002-arquitecto-de-software.md`. Verificado que el archivo resuelto existe en disco (1s confirmó) y que `architect.md` ya no existe bajo ningún path.
- `AGENTS.md` §III y §IV — referencia del Axioma 1 actualizada; tabla de topología reescrita con la ruta real de cada subagente en `.claude/agents/` (6 de 8 roles ya tienen archivo real: `001`, `002`, `003`, `004`, `005`, `008`; `006` / `007` siguen sin subagente propio, marcado explícitamente); §IV-B reescrita para declarar `.agent/` eliminada en vez de seguir apuntando ahí.
- `.claude/agents/002-arquitecto-de-software.md` — `name: architect` (frontmatter) no se había actualizado al renombrar el archivo; corregido a `002-arquitecto-de-software` — sin esto, el identificador interno seguía siendo el viejo pese al `git mv`.
- `.claude/agents/003-esp-diseno-stitch.md` — 1 referencia a `architect.md` corregida.

Validación de gate: `agents/architecture-gate.cjs` pasa `node --check` (sintaxis válida). `--check-gate` corre de punta a punta sin crashear y reporta correctamente `Sin aprobación vigente` — esperado, cualquier cambio de archivo invalida la firma anterior por diseño. No se ejecutó `--aprobar-diseno` (gasta API) sin autorización explícita.

0-I. NOVENA RONDA (2026-08-11, misma jornada) — auditoría forense de `008_AUDITOR_DE_CODIGO` sobre Migración A, hallazgo crítico corregido antes de tocar Supabase

El usuario, correctamente, no aceptó la aprobación del gate de la ronda §0-H como escrutinio real del código de Migración A — esa firma cubrió el trabajo de gobernanza de agentes, sus propias razones dicen explícitamente "no toca... src/modules/... en este diff" pese a que los 3 archivos de Migración A estaban en el mismo diff. Se transfirió el mando a `008_AUDITOR_DE_CODIGO` para una revisión línea por línea, adversarial, de los 3 archivos, antes de autorizar nada contra Supabase.

Hallazgo CRÍTICO — el OCC tal como se construyó no bloquea la colisión que dice prevenir: `occGuard.assertVersionOrConflict()` (lee el último hash vía RPC `obtener_ultimo_hash`) y la escritura (RPC `guardar_modulo10`) eran **2 llamadas PostgREST separadas — 2 transacciones Postgres independientes, sin lock entre ellas**. Bajo concurrencia real (el único caso que le importa a OCC — "modificación concurrente en obra", Fase 4.1 del mandato original), 2 requests simultáneos sobre el mismo proyecto pueden ambos leer el mismo "último hash", ambos pasar el check, y ambos escribir — TOCTOU (time-of-check-to-time-of-use) clásico. El mecanismo "funcionaba" solo en el caso trivial sin concurrencia real, que es precisamente el caso que no necesita protección.

Hallazgo MEDIO — `registrar_version_hash` rompía en guardados sin cambios: un guardado idéntico al anterior produce el mismo hash, chocando con el índice único `idx_pvh_unique_hash` (`duplicate key`, error 500) para un caso que no es un error.

Hallazgo MEDIO, documentado, no "corregible" en esta capa — TRUNCATE bypassa los triggers WORM: `BEFORE DELETE / BEFORE UPDATE` no interceptan `TRUNCATE` en Postgres. Solo explotable con acceso SQL directo (dashboard/administrador), que ya está fuera del perímetro de la aplicación — documentado en el propio archivo de migración para que "WORM" no se lea como garantía absoluta sin matiz.

Verificado y descartado como hallazgo (sin fabricar uno donde no lo hay): los 3 archivos no contienen ninguna operación financiera ni de costos — el requisito de aritmética entera en COP del mandato original no aplica a este diff, se declara explícitamente en vez de forzar un hallazgo falso. Sin riesgo de inyección SQL (parámetros vía RPC/PostgREST, sin concatenación de strings). Aislamiento de tenant correcto en ambas RPCs nuevas.

Corrección aplicada antes de sellar (008 no tiene `Write / Edit` en su propio mandato — la reparación la ejecutó la misma sesión, actuando como el rol con permiso de escritura, `005`, no el auditor):

- `007_worm_occ_shadow_ledger.sql` — `registrar_version_hash` ahora usa `ON CONFLICT (proyecto_id, hash_value) DO NOTHING` + lectura del registro existente — guardado sin cambios deja de ser un error 500.
- `008_occ_atomic_guardar_modulo10.sql` (nueva) — reemplaza `guardar_modulo10: SELECT ... FOR UPDATE` sobre la fila del proyecto (serializa escritores concurrentes del mismo proyecto) + comparación de `p_expected_hash` + `DELETE/INSERT` de indicadores + registro del nuevo hash, **todo en una sola transacción**. Sigue sin `pg`, sin `SET LOCAL app.tenant_id` — restricción dura del ADR-0001 respetada.
- `occGuard.js` y `FormuladorPgController.js` — `guardarModulo10` ya no hace un pre-check en llamada separada; calcula el hash en Node y lo pasa como `p_expected_hash / p_new_hash` a la RPC atómica. `assertVersionOrConflict` / el pre-check separado quedaron retirados del camino de escritura; `obtenerUltimoHash` / `registrarVersionHash` se conservan como utilidades advisory documentadas (ej. UI que avisa "esto cambió" antes de editar), nunca como gate de escritura.

Validación: los 3 archivos JS pasan `node --check . --check-gate` vuelve a reportar (correctamente) que la firma anterior ya no es válida — el nuevo archivo `008_occ_atomic_guardar_modulo10.sql` cambia el listado de `src/`, autoinvalidando la aprobación previa por diseño. No se ejecutó `--aprobar-diseno` (gasta API) sin autorización explícita.

Commit sellado, alcance exclusivamente de este hallazgo (feat(db)):

`src/modules/formulador/migrations/007_worm_occ_shadow_ledger.sql` , `008_occ_atomic_guardar_modulo10.sql` , `occGuard.js` , `FormuladorPgController.js` , `src/shared/infrastructure/validation.js` — el resto de cambios pendientes (gobernanza de agentes, ADR, esta auditoría) queda fuera de este commit a propósito, no se mezclan alcances distintos en un mismo commit.

Sobre "autorización final para ejecutar el SQL en Supabase": el código queda blindado por esta ronda, pero aplicar DDL/triggers nuevos contra la instancia de producción de Supabase es una acción real, de alto impacto y sin deshacer trivial — no se ejecutó de forma autónoma. Los archivos `007` y `008` de `src/modules/formulador/migrations/` están listos para aplicarse (`psql $DATABASE_URL -f ...` o el editor SQL de Supabase, en ese orden), a la espera de confirmación humana explícita para el paso de producción en sí, no solo para el código que lo implementa.

0-J. DÉCIMA RONDA (2026-08-11, misma jornada) — despliegue a Supabase reportado 2 veces, verificado 2 veces, ninguna confirmada; herramienta quirúrgica entregada

Tras el "GO-CODE" de producción (\$0-I), la unidad humana reportó éxito 2 veces al aplicar `007_worm_occ_shadow_ledger.sql` + `008_occ_atomic_guardar_modulo10.sql` vía el editor SQL de Supabase. Ambas veces se verificó por PostgREST (`SUPABASE_URL` / `SUPABASE_SERVICE_KEY` de `.env`, proyecto `ozivmsvxbdtjkzleqbcy`) con las mismas 5 sondas de solo lectura (tabla `project_version_hashes`, tabla `security_violations_ledger`, RPC `obtener_ultimo_hash`, RPC `registrar_version_hash`, RPC `guardar_modulo10` con firma de 5 parámetros) — **resultado idéntico, byte por byte, en ambas corridas:** solo `project_version_hashes` existe; el resto sigue devolviendo `404` (`PGRST205` / `PGRST202`).

Que el segundo barrido diera exactamente igual al primero es la pista, no solo un "sigue fallando" — descarta que haya sido simplemente "faltó ejecutar de nuevo" y apunta a una de 2 causas no distinguibles desde PostgREST hacia afuera: (a) el DDL se está aplicando a una instancia Supabase distinta de la que apunta `.env`, o (b) el caché de esquema de PostgREST no se refrescó tras el DDL.

Entregable quirúrgico: `src/modules/formulador/migrations/_verificar_migracion_a.sql` — script de un solo disparo, para correr DENTRO del propio editor SQL de Supabase (elimina la ambigüedad de "¿es mi vista desde afuera la que está mal, o el DDL nunca llegó?"): consulta `information_schema / pg_proc / pg_trigger` directamente para dar un veredicto OK/FALTA por cada uno de los 5 objetos + los 4 triggers WORM, y termina con `NOTIFY pgrst, 'reload schema';` para forzar el refresco de caché sin costo de volver a aplicar DDL.

Hallazgo técnico adicional, documentado dentro del propio script: `CREATE OR REPLACE FUNCTION` con distinto número de parámetros no reemplaza la función existente en Postgres — la identidad de una función es (nombre + tipos de parámetros), así que la versión nueva de `guardar_modulo10` (5 parámetros) y la vieja (3 parámetros) coexisten como sobrecarga si la migración corre bien, no se sustituyen solas. El script deja una consulta de confirmación y un `DROP FUNCTION` opcional (a ejecutar manualmente, solo después de confirmar que la versión nueva funciona) para no dejar código muerto respondiendo al mismo nombre.

Sin declarar Misión Cumplida todavía — pendiente de que el próximo intento se verifique con esta herramienta dentro del propio Supabase antes de que se reporte como éxito.

Entregable adicional — bloque atómico de despliegue: el patrón "2 pegados separados, 2 estados a medias, cero error visible" se repitió lo suficiente como para dejar de tratarlo como incidente aislado. Se creó `src/modules/formulador/migrations/_deploy_atómico_migracion_a.sql` — combina `007+008` en un único `BEGIN / COMMIT` explícito con 9 puntos de control (`RAISE NOTICE`), incluye el `DROP FUNCTION` de la firma vieja de `guardar_modulo10` (3 parámetros) que `CREATE OR REPLACE` con distinto número de argumentos no elimina por sí solo (identidad de función en Postgres = nombre+tipos de parámetros, no un simple reemplazo), y termina con la verificación de los 8 objetos + `NOTIFY pgrst, 'reload schema'` en la misma pantalla.

Institucionalizado, no solo parcheado esta vez (a pedido explícito del usuario — "esto debe estar entre las habilidades del orquestador o arquitecto"): `.claude/agents/005-ingeniero-backend.md` ahora tiene una regla permanente — todo DDL que entregue de aquí en adelante debe venir en bloque atómico con esas mismas 5 propiedades (transacción explícita, checkpoints, idempotencia, `DROP` de overloads viejos, verificación inline). `.claude/agents/002-arquitecto-de-software.md` suma un 6º criterio de evaluación: no aprobar una propuesta de DDL que no cumpla ese formato — la responsabilidad de blindar el próximo despliegue no depende de que se repita el mismo incidente para que alguien se acuerde de exigirlo.

0-K. UNDÉCIMA RONDA (2026-08-11, misma jornada) — despliegue parcial verificado con evidencia nueva, causa raíz distinta a las rondas anteriores

Cuarto reporte de "éxito" del despliegue en Supabase. Verificado por API (mismas 5 sondas de siempre) — esta vez el resultado **sí cambió**, a diferencia de los 2 intentos anteriores (byte por byte idénticos): `security_violations_ledger` ya existe (antes `404`). Progreso real, no más un estado congelado.

Pero apareció un hallazgo nuevo, más sutil que "no se aplicó nada": `guardar_modulo10` (5 parámetros) ahora **existe y responde**, pero falla en tiempo de ejecución con `42703: column "version_hash" does not exist` — un error que no puede venir del código auditado (ningún archivo de este proyecto declara una columna con ese nombre; `version_hash` solo existe como clave de un JSON de retorno y como campo del body en Node, nunca como columna SQL). `obtener_ultimo_hash` sigue en `404`, pero esta vez con una pista reveladora: PostgREST sugiere una función existente `obtener_ultimo_hash(p_project_id)` — un solo parámetro, en inglés, distinto a la firma real (`p_tenant_id`, `p_proyecto_id`).

Causa raíz identificada: hay versiones divergentes de estas funciones ya viviendo en la base — de un intento anterior, una edición manual, o una mezcla de borradores no confirmada. `CREATE OR REPLACE FUNCTION` en Postgres identifica una función por nombre **más tipos de parámetro**, no por archivo de origen ni por nombre de parámetro — si la firma divergente no coincide exactamente con la de este proyecto, `CREATE OR REPLACE` crea un **overload** nuevo y dejaba la versión vieja (rota) respondiendo al mismo nombre, invisible hasta que se prueba en vivo.

Fix aplicado a `_deploy_atomico_migracion_a.sql` antes de un nuevo intento (no se adivinó la firma divergente, se eliminó la clase completa del problema): nuevo **BLOQUE 0**, al inicio de la transacción, que consulta `pg_proc` dinámicamente y elimina (`DROP FUNCTION`) **todos** los overloads existentes de las 4 funciones RPC de Migración A, sin importar su firma, antes de recrear la versión canónica. Ya no depende de que alguien identifique manualmente qué firma vieja quedó viva.

Sin declarar éxito todavía — pendiente de que se corra la versión actualizada del script (con el Bloque 0 de limpieza) y se verifique de nuevo.

0-L. DUODÉCIMA RONDA (2026-08-12) — Migración A confirmada en Supabase, causa raíz cerrada con evidencia

Tras 7 rondas de "éxito reportado, verificación en rojo", la causa se identificó y se cerró: quedaba una tabla `project_version_hashes` con un esquema divergente (mucho más simple que las 11 columnas de este diseño — confirmado por un error real de Postgres: `null value in column "hash_value"`, fila fallida con solo 3 valores) de algún intento previo. `CREATE TABLE IF NOT EXISTS` nunca la tocaba porque, técnicamente, ya existía — solo con la forma equivocada. Se agregó `DROP TABLE IF EXISTS project_version_hashes,` `security_violations_ledger CASCADE` (confirmadas vacías por API antes de borrar, cero riesgo de datos) al inicio de `_deploy_atomico_migracion_a.sql`, además del **BLOQUE 0** de la ronda anterior (limpieza dinámica de overloads de función). Commit `f63cfdc`.

Barrido final, verificado por API, no solo reportado:

Objeto	Resultado
<code>project_version_hashes</code>	✅ 200
<code>security_violations_ledger</code>	✅ 200
<code>obtener_ultimo_hash</code>	✅ 200, retorna <code>{"hash_value": null, "created_at": null}</code> — comportamiento exacto de la función para un proyecto sin hash previo
<code>registrar_version_hash</code>	✅ Rechaza correctamente un proyecto inexistente (<code>P0001</code> , mismo mensaje del <code>RAISE EXCEPTION</code> del código fuente)
<code>guardar_modulo10</code> (5 parámetros)	✅ Misma validación correcta, mismo mensaje

Sin probar todavía (no forzado a propósito, para no ensuciar producción): que el trigger WORM bloquee un `UPDATE / DELETE` real, y el comportamiento de OCC bajo una colisión real con datos existentes — ambos se validan naturalmente en el primer uso real del módulo Formulador, no requieren una prueba sintética contra producción.

Migración A: cerrada. Fases 2.2 y parte de 3 (guardrail de tenant, base de cálculo COP) ya estaban. Fase 1 (WORM) y Fase 4.1/4.2 (OCC atómico, Shadow Ledger) ahora tienen estructura real en la base de datos viva, no solo en el código. Migración B (RLS real) sigue bloqueada por ADR-0001, pendiente de que el usuario confirme Third-Party Auth en Supabase.

0-M. AUDITORÍA ENFOCADA — AGENTES 001-006 (2026-08-12)

Repetición del protocolo forense original, esta vez con alcance explícito del usuario limitado a los roles `001_ORQUESTADOR_MAESTRO` a `006_DEVSECOPS_INFRAESTRUCTURA` (excluye `007 / 008`). Cada hallazgo se releyó en disco en esta misma ronda, no se heredó de memoria de rondas anteriores — donde algo no cambió desde \$0-H/\$0-L se dice explícitamente "sin cambio, re-verificado hoy".

0-M.1 Inventario total y organigrama jerárquico (001-006)

Rol	Archivo real	Tools	¿Subagente ejecutable o etiqueta?
001_ORQUESTADOR_MAESTRO	.claude/agents/001-orquestador-maestro.md (creado por actor externo a esta sesión, corregido 2026-08-11)	Read, Grep, Glob, Bash, Write, Edit, Agent, WebSearch, WebFetch	● Real — único punto de contacto declarado con el usuario, tabla de ruteo a 002-008
002_ARQUITECTO_DE_SOFTWARE	.claude/agents/002-arquitecto-de-software.md	Read, Grep, Glob (solo lectura)	● Real — único con gate técnico verdadero: invocado por agents/architecture-gate.cjs --aprobar-diseno (API Anthropic real), firma SHA-256 autoinvalidante, obligatorio vía .git/hooks/pre-commit
003_ESP_DISENO_STITCH	.claude/agents/003-esp-diseno-stitch.md	Read, Grep, Glob	● Real — solo lectura, audita tokens Tailwind/estilos, sin conexión a herramientas mcp__stitch__* (fuera de su alcance por diseño)
004_SENTINELA_FRONTEND	.claude/agents/004-sentinela-frontend.md	Read, Grep, Glob	● Real — solo lectura, detecta stubs huérfanos (FrozenPage.jsx) y contratos de build rotos
005_INGENIERO_BACKEND	.claude/agents/005-ingeniero-backend.md	Read, Write, Edit, Grep, Glob, Bash	● Real — único de los 6 con permiso de escritura, mayor blast radius del grupo, gobernado por docs/ADR/ADR-0001-auth-r1s-worm-occ.md
006_DEVSECOPS_INFRAESTRUCTURA	No existe — confirmado con ls .claude/agents/ en esta misma ronda	—	● Etiqueta pura — solo vive como entrada en ESCUADRON_ELITE (agents/architecture-gate.cjs:47-53) y en AGENTS.md

Árbol jerárquico (fuente: agents/architecture-gate.cjs:29-63 , cruzado con .claude/agents/*.md):

```
001_ORQUESTADOR_MAESTRO (.claude/agents/001-orquestador-maestro.md)
|  Único punto de contacto con el usuario – enruta, no ejecuta código operativo.
|
|— 002_ARQUITECTO_DE_SOFTWARE (.claude/agents/002-arquitecto-de-software.md)
|   Sin subordinados en ESCUADRON_ELITE – es un gate transversal, no un nodo
|   de dominio. Invocado por agents/architecture-gate.cjs antes de cualquier
|   commit (.git/hooks/pre-commit + --check-gate).
|
|— 003_ESP_DISENO_STITCH (.claude/agents/003-esp-diseno-stitch.md)
|   subordinados: [] – nunca tuvo carpeta propia en agents/
|
|— 004_SENTINELA_FRONTEND (.claude/agents/004-sentinela-frontend.md)
|   subordinados: [] – mismo patrón que 003
|
|— 005_INGENIERO_BACKEND (.claude/agents/005-ingeniero-backend.md)
|   |— 009_gestor_datos
|   |— 011_Radar1_minero
|   |— 012_Radar2_Estratega
|   |— 050_Formulador_proy
|   |— 07-ing-concreto_GFRC (fuera de dominio Formulador/Radar)
|   |— 08-estratega-neuromarketing (fuera de dominio Formulador/Radar)
|
|— 006_DEVSECOPS_INFRAESTRUCTURA [SIN SUBAGENTE REAL]
|   |— 03-analista-secop
|   |— 052_Form_Administrativo
|   |— 14-analista-comportamiento
|   |— 015_intelligence-core
```

Desalineaciones de rol detectadas:

- 006 es el único de los 6 sin ningún archivo .claude/agents/*.md — su rol1 declarado ("Despliegues a producción, servidores, fiscalización de seguridad... COP") no coincide con ninguno de sus 4 subordinados reales (03-analista-secop , 052_Form_Administrativo , 14-analista-comportamiento , 015_intelligence-core — ninguno hace despliegues ni gestiona servidores). Ya documentado como mandato pendiente en AGENTS.md:37 y [[project_006_devsecops_pendiente]] (memoria persistente, 2026-08-11) — no repetido aquí como hallazgo nuevo, solo re-confirmado vigente.
- ¿Falta un nodo orquestador central? No — 001_ORQUESTADOR_MAESTRO cumple ese rol y tiene archivo real, a diferencia de la auditoría original (2026-08-08) donde este nodo no existía como subagente ejecutable de Claude Code, solo como convención en IDENTITY.md .

3. ¿Agentes ejecutores operando sin validación previa? No entre 001-006: 005 (el único con Write / Edit) tiene instrucción explícita de invocar a 002 antes de tocar subsistemas nuevos (WORM/OCC), y esa regla se demostró cumplida en la práctica esta misma sesión (ADR-0001). El riesgo real no es ausencia de regla, es que el enforcement depende de que 005 la respete por convención — el frontmatter de Claude Code no tiene forma técnica de impedir que un agente con Write ignore una instrucción de texto (ya documentado en 005-ingeniero-backend.md:17).

Clasificación transversal vs. específico:

- **Transversales (gobiernan todo el proyecto, no un dominio):** 001 (enrutador), 002 (gate de arquitectura).
- **Específicos por capa del proyecto:** 003 / 004 (frontend/SPA), 005 (persistencia/backend), 006 (infraestructura — hoy sin implementación).

0-M.2 Auditoría anatómica y forense de skills (001-006)

001 y 002 no tienen skills propias en el sentido de agents/*/skills/*.cjs — su lógica vive íntegra en el prompt del archivo .md (comportamiento de LLM, no código ejecutable independiente). 003 / 004 tampoco (solo lectura vía herramientas nativas de Claude Code). El código real vive bajo los subordinados de 005 y 006 :

Skill	Bajo	Función real (I/O)	Manejo de errores	Veredicto (re-confirmado hoy, ver §2.3 para el detalle original)
Skill_001_Fix-Encoding.cjs	005 → 009_gestor_datos	Corrige acentos rotos a entidades HTML	try/catch , retorna false en error	● Real, funcional
Skill_001_Gestor-Encoding.cjs	005 → 009_gestor_datos	Igual + modo check	Parcial	● Riesgo sin resolver: execSync('firebase deploy --only hosting') activable con --deploy , sin gate de confirmación (plan de remediación ítem 14, sigue pendiente)
Skill_001_OCR_Soporte.cjs	005 → 009_gestor_datos	Solo lee extensión de archivo, cero OCR real	N/A	● Nombre engañoso, sin corregir
radar_oficial.py / test_fuentes.py	005 → 011_Radar1_minero	Scraping real (requests + BeautifulSoup)	try/except presente	● Técnicamente competente, rastrea 6 países no-Colombia (contradice Axioma II.2 de AGENTS.md , ítem 18 del plan de remediación, sin resolver)
Skill_050_Formulador_Proyecto.cjs	005 → 050_Formulador_proy	Generador de plantillas boilerplate, no formula nada	N/A	● Cita Skill_057_Interventor , rol que no existe en la numeración actual
Skills bajo 03-analista-secop , 052_Form_Administrativo , 14-analista-comportamiento , 015_intelligence-core	006 (huérfano)	Sin dueño real que las audite — 006 no tiene subagente que revise sus propios subordinados	—	● Brecha de gobernanza: estos 4 subordinados existen en disco pero ningún subagente real los fiscaliza (a diferencia de los de 005 , que sí tiene dueño ejecutable)

Escaneo de anomalías (001-006 específicamente):

- **Injerto confirmado y ya corregido esta sesión:** .claude/agents/001-orquestador-maestro.md fue agregado por un actor externo a esta sesión con 2 defectos reales — routing a 004_INGENIERO_FRONTEND (nombre retirado) y una sección "Agentes de Apoyo 06X" que reintroducía la clasificación ya corregida en la ronda §0-B, apuntando a una carpeta (.agent/) que para ese momento ya había sido destruida. Corregido en §0-H, verificado de nuevo hoy: grep -n "004_INGENIERO_FRONTEND\|06X" .claude/agents/001-orquestador-maestro.md → sin coincidencias, limpio.
- **Sin código huérfano nuevo detectado en 002-005** en esta ronda — los 4 archivos son internamente consistentes entre sí (cross-referencian nombres de archivo correctos: 002-arquitecto-de-software.md , 003-esp-diseno-stitch.md , 004-sentinel-a-frontend.md , 005-ingeniero-backend.md , sin ninguna mención residual a architect.md).
- **Duplicidad de habilidad no resuelta:** 003_ESP_DISENO_STITCH y 004_SENTINELA_FRONTEND ambos detectan "componentes huérfanos"/"stubs" desde ángulos distintos (visual vs. datos) — el propio archivo de 003 ya declara la coordinación explícita ("si el stub ya está documentado por 004, no lo reportes de nuevo") para prevenir que se cuente doble, pero es una instrucción de prompt, no un mecanismo técnico — mismo patrón de "restricción de honor" que el resto del sistema.

0-M.3 Mapa de integraciones, flujos y comunicaciones (001-006)

Único flujo con integración real de código verificable, sin cambios desde \$0-D:

```
usuario → 001 (enrutamiento por prompt, sin código)
  → 005 propone tocar WORM/OCC
  → 002 evalúa (agents/architecture-gate.cjs → API Anthropic real,
    system prompt = 002-arquitecto-de-software.md, input = git diff)
  → veredicto {"aprobado": bool} → agents/disenso_aprobado.json
  → .git/hooks/pre-commit bloquea el commit si la firma no es vigente
```

Este es el único punto donde "agente → backend/API externa" es código real, no solo prompt — todo lo demás en 001-006 es razonamiento de LLM sobre Read/Grep/Glob.

SPOF identificado: la firma de `agents/disenso_aprobado.json` se invalida por **lista de archivos** en `agents/ + src/ (hashEstado() en architecture-gate.cjs)`, no por contenido — un cambio de contenido en un archivo ya existente (sin agregar/quitar ninguno) **no invalida la firma**. Confirmado empíricamente esta sesión: varios commits de `.claude/agents/*.md` pasaron el gate sin pedir nueva aprobación porque no cambiaron la lista de archivos de `src/`. Es una brecha real del propio gate de `002` — el mecanismo que se supone impide "código sin diseño aprobado" tiene un punto ciego ante ediciones de archivos ya existentes.

Brecha de estructuración sin aprobar, específica de 006: como `006` no tiene subagente real, cualquier tarea de "despliegue/infraestructura" delegada por `001` (según su propia tabla de ruteo, `001-orquestador-maestro.md:26`) no tiene a quién llegar — cae por defecto en el agente ejecutor genérico (Claude principal), sin el filtro de dominio que si tienen `003 / 004 / 005`. No hay pérdida de contexto detectada en las transiciones `001→002` y `001→005` (ambas dejan rastro escrito: `disenso_aprobado.json`, y los propios archivos `.md` de cada agente documentan el estado real del proyecto para que la siguiente invocación no repita trabajo) — pero `001→006` no puede dejar rastro porque no hay receptor.

0-M.4 Análisis de límites, bloqueos y gaps — expectativa vs. realidad (001-006)

Agente	Promesa	Realidad verificada hoy
001	"Tolerancia cero ante violaciones de arquitectura, aislamiento (RLS) o moneda (COP)"	Enrutador de prompt — no tiene mecanismo propio de bloqueo técnico; la tolerancia cero real la ejecuta <code>002</code> (gate) y el guardrail de <code>assertValidTenant()</code> en código, no <code>001</code>
002	"Cero código sin diseño aprobado"	● Cumplida con mecanismo técnico real (hook + API), con el punto ciego de <code>hashEstado()</code> ya señalado en \$0-M.3
005	"Aislamiento multi-tenant... Fase 2.1 cero <code>service_role</code> en lógica de negocio regular"	● Contradice el comportamiento actual documentado en su propio archivo (<code>supabaseClient.js</code> degrada a <code>SERVICE_KEY</code> siempre) — declarado explícitamente como contradicción conocida, no oculta
006	"Despliegues a producción, servidores, fiscalización de seguridad... COP"	● No existe ejecución alguna — ni despliegues, ni fiscalización, ni nada. 100% aspiracional

Regla de negocio COP (Axioma II.2 de AGENTS.md): entre 001-006, el único punto con cálculo financiero real es bajo `005` (`AGT-053` en `src/orchestrator-engine.js`, AIU+IVA en COP, sin conversión de divisas) — cumple la regla en el único lugar donde aplica. `011_Radar1_minero` (bajo `005`) sigue rastreando 6 países no-Colombia, dato geográfico no financiero pero sí fuera del foco nacional declarado (Axioma II.2, ítem 18 del plan de remediación, sin resolver desde la auditoría original).

Aislamiento de estado por usuario: cumplido a nivel de código (`assertValidTenant()`, filtro `WHERE tenant_id` explícito en cada RPC de `005`) pero no a nivel de RLS real de Postgres — ver ADR-0001, Migración B sigue bloqueada por falta de Third-Party Auth (acción humana pendiente, no de ningún agente 001-006).

0-M.5 Plan de remediación y blindaje estructural (001-006)

#	Hallazgo	Agente	Criticidad	Estado
1	<code>006_DEVSECOPS_INFRAESTRUCTURA</code> sin subagente real, rol declarado no coincide con subordinados	006	● Alta	Pendiente — mandato ya anotado en <code>AGENTS.md:37</code> y memoria persistente, no construido todavía
2	<code>hashEstado()</code> del gate no detecta cambios de contenido en archivos ya existentes (solo altas/bajas)	002	● Media	Pendiente — no reportado en rondas anteriores, hallazgo de esta ronda
3	<code>Skill1_001_Gestor_Encoding.cjs</code> dispara <code>firebase deploy</code> sin gate	005 (subordinado <code>009_gestor_datos</code>)	● Alta	Pendiente desde \$2.3, sin cambio
4	<code>011_Radar1_minero</code> rastrea 6 países no-Colombia	005 (subordinado)	● Media-baja	Pendiente, decisión de alcance sin tomar

5	Coordinación 003 ↔ 004 es solo instrucción de prompt, sin mecanismo técnico que impida doble reporte	003, 004	Baja	Aceptable — bajo impacto, ambos son de solo lectura
6	001→006 no tiene receptor real cuando se delega infraestructura	001, 006	Media	Depende de #1 — se resuelve cuando 006 exista

Diseño del Agente de Arquitectura de Software — ya no es un diseño pendiente, es el hallazgo positivo central de esta auditoría enfocada: a diferencia del prompt original (que asumía la ausencia de este agente como brecha), 002_ARQUITECTO_DE_SOFTWARE existe, tiene mecanismo técnico real (`.git/hooks/pre-commit + agents/architecture-gate.cjs --check-gate` , sin costo de API por commit; `--aprobar-diseno` invoca la API real de Anthropic solo cuando hace falta nueva firma), y se demostró funcionando en múltiples ciclos reales durante esta sesión (ADR-0001, Migración A). El único gap real de diseño que le queda es el de `hashEstado()` señalado en el ítem 2 — no la ausencia del agente.

0-N. CIRUGÍA ESTRUCTURAL 001-005 (2026-08-12) — 5 objetivos de remediación

Ejecución directa de los 5 hallazgos del plan de remediación de \$0-M.5, sin volver a auditar (ya estaba hecho):

- hashEstado() reescrita** (`agents/architecture-gate.cjs`) — ahora hashea CONTENIDO de archivo (`crypto.createHash('sha256').update(fs.readFileSync(...))`), no solo nombres. Ampliado además a `.claude/agents/*.md` , que antes no estaba en el alcance de la firma en absoluto — se podía reescribir el prompt completo de cualquier agente sin invalidar nunca el gate.
- Trigger de despliegue eliminado** — `agents/009_gestor_datos/skills/Skill_001_Gestor_Encoding.cjs` ya no tiene `deploy()` , `execSync` , ni el flag `--deploy` . Verificado con `grep`: cero coincidencias.
- Skill_Soporte_Automatico.cjs purgado** (`git rm agents/010_redactor_tecnico/skills/Skill_Soporte_Automatico.cjs`), y su entrada huérfana eliminada de `skills/ag_skills_registry.json` (JSON re-validado tras el cambio).
- radar_oficial.py restringido a Colombia** — `FUENTES_ABIERTAS` pasó de 6 fuentes extranjeras (Costa Rica, Chile, Argentina, Uruguay, Paraguay, Panamá) a 1 sola: SECOP II, reutilizando la URL ya validada en `test_fuentes.py` del mismo directorio (no se inventó una URL nueva). Extracción de presupuesto reescrita de `presupuesto_usd` (patrones USD/EUR) a `presupuesto_cop` (formato COP, punto como separador de miles). `test_fuentes.py` no requería cambio — ya solo tenía SECOP Colombia + 2 organismos multilaterales (BID, UNDP), ninguno de los 6 países señalados.
- No aplicaba** — verificado con `grep` que `100_reparador_codigo / 09-legal-licitaciones` ya no existían en `IDENTITY.md` , resuelto en una ronda anterior.

Validación: sintaxis verificada en los 4 archivos tocados (`node --check` ×3, `py_compile` ×1, JSON re-parseado). `--check-gate` reporta correctamente que la firma anterior ya no es válida — la reescritura de `hashEstado()` invalida cualquier firma previa por diseño, además de los cambios reales de contenido. No se ejecutó `--aprobar-diseno` (gasta API) sin autorización explícita.

0-O. HALLAZGO Y FIX — 001 con permiso de escritura pese a mandato de solo-enrutamiento (2026-08-12)

Pregunta exploratoria del usuario ("cómo subir la eficacia del Escuadrón 001-005") reveló la inconsistencia de mayor impacto detectada hasta ahora: 001_ORQUESTADOR_MAESTRO tenía `Write` , `Edit` y `Bash` directos en su frontmatter (`.claude/agents/001-orquestador-maestro.md`) pese a que su propio \$1 dice "tu función NO es escribir código operativo". Era el único agente del Escuadrón capaz de mutar el repositorio sin pasar por el único punto de enforcement técnico real del sistema (`.git/hooks/pre-commit + agents/architecture-gate.cjs`) — 002 / 003 / 004 no tienen permiso de escritura en absoluto, y 005 (el único que sí lo tiene) está explícitamente instruido a invocar a 002 antes de tocar subsistemas nuevos.

Fix: `tools` de 001 reducido a `Read` , `Grep` , `Glob` , `Agent` , `WebSearch` , `WebFetch` — sin `Write` / `Edit` / `Bash` . Documentado en el propio archivo (\$4 nueva) el porqué, para que no se reintroduzca por error en una futura edición. 001 queda forzado a delegar vía `Agent` a 002 - 005 para cualquier tarea que mute el repo, en vez de poder ejecutar por fuera del gate.

0-P. BLINDAJE "RELOJ SUIZO" 001-005 (2026-08-12) — 3 fixes estructurales

Ejecución de los 3 hallazgos de mayor severidad del reporte élite de esta misma fecha (hook no versionado, cero tests del gate, 003/004 sin enforcement técnico). El 4º (auto-aprobación sin separación de funciones) queda documentado como riesgo aceptado, no implementado — es una decisión de proceso, no de código.

1. Hook versionado. `scripts/pre-commit.sh` (lógica real, trackeada por git) + `scripts/install-git-hooks.cjs` (instalador cross-platform) + "prepare": "node scripts/install-git-hooks.cjs" en `package.json` (corre automático en `npm install`). `.git/hooks/pre-commit` es ahora un wrapper de 2 líneas que invoca al script versionado — verificado funcionando end-to-end (`--check-gate` sigue bloqueando correctamente vía el wrapper nuevo).

2. Tests del gate. `scripts/architecture-gate.test.cjs` (Node nativo, `node:test`, cero dependencias nuevas) — 10 casos, cubren exactamente la clase de bug que ya se coló una vez sin detectarse (contenido vs. nombre de archivo), más el mecanismo de subgates nuevo. Requirió un cambio estructural en `agents/architecture-gate.cjs`: se agregó guardia `if (require.main === module)` alrededor de `ejecutarTodosLosAgentes()` y `module.exports` — antes, un simple `require()` del archivo (como hace un test) disparaba el batch executor completo de verdad, sin ningún guardia. `npm run test:gate` para correrlos.

3. Subgates elite para 003/004. Mecanismo genérico en `architecture-gate.cjs` (objeto `SUBGATES`) que engancha cualquier agente de solo lectura al mismo `--check-gate` obligatorio que ya protegía solo a `002`. Si un commit toca `src/**/*.jsx|tsx`, `--check-gate` ahora exige un veredicto vigente de `004_SENTINELA_FRONTEND` (`agents/veredicto_004.json`) y `003_ESP_DISENO_STITCH` (`agents/veredicto_003.json`), cada uno con su propio campo de aprobación (`limpio / disenio_valido`) y firma de hash sobre el contenido staged relevante — mismo patrón que `002`, pero paramétrico. Nuevo modo `--aprobar-subgate <agentId>` invoca la API real de Anthropic con el prompt de ese agente. Diseñado para que agregar un futuro agente "elite" sea una entrada nueva en `SUBGATES`, no una reescritura** — respuesta directa al pedido del usuario de que "los demás agentes que se agreguen estén al nivel Elite".

Validación: `node --check limpio`, `npm run test:gate` → 10/10, `--check-gate real` verificado (bloquea correctamente en el gate de `002`, que corre primero por diseño). No se ejecutó `--aprobar-diseno` ni `--aprobar-subgate` (gastan API) sin autorización.

0-Q. PUESTO DE MANDO UNIFICADO (PMU) — 2026-08-12

A pedido explícito del usuario ("control absoluto... que sea escalable para futuros agentes"), se construyó el PMU real sobre el gate ya probado, no un sistema paralelo:

- `agents/pmu/estado_operativo.json` — foto única del escuadrón completo, generada por código. Auto-descubre agentes vía `descubrirAgentes()` leyendo `.claude/agents/*.md` (parseo minimalista de frontmatter `name / tools`, sin dependencia YAML) — un agente nuevo aparece en el tablero sin tocar código, que es la respuesta directa a "escalable para futuros agentes".
- `agents/pmu/telemetria.jsonl` — append-only, una línea por cada decisión real de gate (`--check-gate`, `--aprobar-diseno`, `--aprobar-subgate`), con tipo/subsistema/resultado/razón/timestamp. Antes no había manera de responder "¿cuántas veces bloqueó el gate?" sin leer la narrativa a mano.
- `--pmu-status` — comando nuevo, sin costo de API, imprime el tablero legible por humano en el momento.

Verificado funcionando de verdad, no solo compilando: corrí `--pmu-status` contra el estado real del repo — detectó los 6 agentes existentes (001-005, 008), marcó correctamente que `001` ya no tiene permiso de escritura (fix de la ronda anterior) y que `005 / 008` sí lo tienen.

Hallazgo que el propio tablero sacó a la luz, no buscado a propósito: `008_AUDITOR_DE_CODIGO` tiene `Bash` en su frontmatter pese a que su mandato dice "NO ESCRIBE CÓDIGO NUEVO" — a diferencia del caso de `001` (que no tenía ningún uso legítimo para esas herramientas), `008` sí podría necesitar `Bash` para su rol de QA/Red Team (correr pruebas, intentar exploits) — no se tocó sin confirmar la intención real, queda señalado, no corregido unilateralmente.

Fuera de alcance de esta ronda, a propósito (no es "una entrada de config más"): un agente que vigile `telemetria.jsonl` y proponga fixes sin intervención humana, y un protocolo estructurado de mensajería entre agentes (hoy siguen coordinando por prosa en su system prompt, no por mensajes con confirmación de recibido). Señalados como siguiente fase si se quiere llevar el PMU más allá de estado+telemetría.

Validación: `node --check limpio`, `npm run test:gate` → 14/14, `--pmu-status` corrido en vivo contra el repo real (no un mock), `estado_operativo.json` real generado y commiteado como evidencia del primer corte.

0-R. `006_DEVSECOPS_INFRAESTRUCTURA` CONSTRUIDO — perfil élite completo (2026-08-12)

A pedido del usuario ("no te parece que le falta mucho" tras una primera propuesta demasiado tímida), se amplió el mandato antes de construir: no solo auditar `render.yaml` / `npm audit` bajo demanda, sino cerrar 3 brechas de seguridad reales que esta misma sesión ya había descubierto y dejado sin dueño (secretos filtrados, `.env.example` inexistente, dependencias nunca auditadas).

Subagente real: `.claude/agents/006-devsecops-infraestructura.md` — `tools`: Read, Grep, Glob, Bash, sin Write / Edit (auditor, no ejecutor). Documenta explícitamente que NO debe tocar los 4 subordinados huérfanos sin que se le pida (orden del usuario: esa limpieza espera a que el Escuadrón esté completo, ver memoria persistente).

Chequeos deterministas nuevos en `agents/architecture-gate.cjs`, sin costo de API, corren en TODO `--check-gate`:

1. **Escaneo de secretos** (escanearSecretos) — patrones específicos de bajo falso-positivo (JWT de 3 segmentos, AWS Access Key, bloque PEM de llave privada, asignación `key=valor` larga) sobre el contenido staged real (`git show :archivo`), no el de disco. Bloquea `.env` real por nombre de archivo, sin falta leer contenido.

2. `.env.example` (verificarEnvExample) — **no existía en la raíz del proyecto**, confirmado y corregido en esta misma ronda: se generó con las 28 variables reales de `.env`, solo nombres. El chequeo ahora compara nombres y bloquea si se desincroniza en el futuro.

3. `npm audit` (verificarDependencias) — solo corre cuando `package.json` / `package-lock.json` está en el diff (no en cada commit, evita depender de red en cada commit sin relación). **Hallazgo real, no hipotético:** al probarlo contra las dependencias reales de este proyecto, encontré **4 vulnerabilidades críticas, 20 altas, 21 moderadas, 2 bajas** — nadie había corrido `npm audit` en ningún punto de esta sesión hasta ahora. No se aplicó ningún fix automático (`npm audit fix` puede romper builds, requiere autorización explícita del usuario, mismo criterio que toda acción de alto impacto de esta sesión).

PMU: `006-devsecops-infraestructura` aparece solo en `--pmu-status` (auto-discovery, cero código nuevo necesario) — confirmado en vivo, 7 de 8 agentes ahora tienen subagente real (falta solo `007`).

Validación: `node --check` limpio, `npm run test:gate` → **18/18**, `--pmu-status` y `--check-gate` corridos en vivo contra el repo real. Corrección de un bug real encontrado en el camino: `npm.cmd` en Windows requiere `shell:true` para ejecutarse (es un batch script, no un binario nativo) — args siempre literales fijos, nunca input externo, así que la advertencia `DEP0190` de Node no aplica al riesgo que señala.

0-S. ROSTER COMPLETO — 8 de 8, + CI real (2026-08-12)

007_DOCUMENTADOR_AS_BUILD construido: `.claude/agents/007-documentador-as-build.md`, tools: Read, Write, Edit, Grep, Glob (blast radius limitado a `docs/`). Su mandato original (artefacto atado a `ejecutarTodosLosAgentes()`, el batch executor legado — nunca corrió de verdad, `docs/as-build/` no existe en disco) se amplió a lo que en la práctica ya se venía haciendo toda esta sesión: mantener este mismo documento como registro as-built vivo. **Con esto, los 8 roles del Escuadrón Élite tienen subagente real — roster completo.**

Fase 3 de la estrategia 100/100 — CI real, no solo el hook local: `.github/workflows/gate.yml` — corre `npm run test:gate` en cada push/PR a `master` / `main`, sin costo de API (no invoca `--aprobar-diseno`). Cierra la brecha de que el hook de pre-commit, aunque ya versionado (`$0-P`), solo protege a quien lo tiene instalado — un clone sin `npm install`, o un commit con `--no-verify`, no pasa por él. El paso de `npm audit` queda con `continue-on-error: true` a propósito — las 4 críticas/20 altas de `$0-R` siguen sin resolver, bloquear el CI por eso ahora mismo pararía todo el equipo sin que nadie lo haya decidido explícitamente. Instrucción dejada en el propio workflow: quitar `continue-on-error` cuando se resuelvan.

Hallazgo de precisión, encontrado al sellar esta misma ronda: el chequeo de dependencias de `006` disparaba con solo que `package.json` estuviera en el diff, sin importar qué cambiara adentro — bloqueó un commit que solo agregaba 2 scripts npm, sin tocar ninguna dependencia. Corregido (`diffTocaDependencias()`): ahora inspecciona el diff staged real y solo aplica si un renglón dentro de `dependencies` / `devDependencies` cambió de verdad.

Pendiente explícito, no resuelto a propósito (orden del usuario, 2026-08-12): limpieza/reasignación de los subordinados huérfanos de `005` y `006` — se aplaza hasta este punto (roster completo), que es exactamente donde estamos ahora. Sigue en memoria persistente, sacar a relucir en la próxima ronda.

Validación: `npm run test:gate` → 19/19, `--pmu-status` confirma 8/8 agentes con subagente real, `--check-gate` pasa limpio tras el fix de precisión, YAML del workflow validado. Commit `c89c54f` sella todo lo de esta ronda hasta `006`; `007` + CI se commitean aparte a continuación.

0-T. CIERRE DE LOS 4 PENDIENTES HUMANOS + LIMPIEZA DE SUBORDINADOS (2026-08-12)

3 de 4 pendientes verificados de forma independiente, no solo aceptados por reporte:

1. `npm audit fix` (manual, sin `--force`): verificado con `npm audit --json` real — bajó de 47 a **23 vulnerabilidades** (2 críticas, 7 altas, 14 moderadas, 0 bajas). Coincide exacto con lo reportado. Correcto no forzar — `--force` de npm aplica bumps de versión mayor que pueden romper el build sin aviso.

2. **JWT `service_role` de Supabase:** revocación reportada desde el dashboard. **No verificable por mí** — no tengo forma de confirmar el estado de una key desde fuera del dashboard de Supabase. Se acepta el reporte, sin evidencia independiente (a diferencia de los otros 3 puntos).

3. **Rama de despliegue en Render:** `origin/radfor360-production` — confirmado que la rama **existe de verdad** en el remoto (`git fetch` + `git branch -a`). Cuál rama usa Render en el dashboard sigue sin ser verificable desde disco, pero la existencia de la rama que el usuario nombró es real, no inventada.

4. **Limpieza/reasignación de subordinados de `005` / `006`:** autorizada y ejecutada esta misma ronda (detalle abajo).

Limpieza de subordinados — ejecutada

- **Purgados** (`git rm` , vacíos, cero código, mismo criterio que `Skill_Soporte_Automatico.cjs`): `agents/03-analista-secop` , `agents/14-analista-comportamiento` .
- **Reasignados** de `006_DEVSECOPS_INFRAESTRUCTURA` a `005_INGENIERO_BACKEND` en `ESCUADRON_ELITE` (`agents/architecture-gate.cjs`): `052_Form_Administrativo` , `015_intelligence-core` — su contenido real (Formulador, gestión de proyectos de construcción/SECOP) nunca fue infraestructura de despliegue.
- `006_DEVSECOPS_INFRAESTRUCTURA` **queda con subordinados**: `[]` — ya no agrupa carpetas legacy ajenas a su dominio; su trabajo vive en su propio subagente (`.claude/agents/006-devsecops-infraestructura.md`) y en los chequeos deterministas del gate. Esto cierra, por fin, la desalineación de rol que la auditoría original señaló en §1.4 (2026-08-08): " `006` dice hacer despliegues, ninguno de sus subordinados lo hacía".
- Referencias colgantes corregidas: `skills/ag_skills_registry.json` (2 entradas huérfanas eliminadas), `agents/001_ORQUESTADOR_MAESTRO/IDENTITY.md` (tabla de ruteo ya no cita `03-analista-secop`), `.claude/agents/006-devsecops-infraestructura.md` (nota de origen actualizada de "pendiente" a "resuelto").

Validación: `node --check` limpio, `npm run test:gate` → 19/19, `--pmu-status` sigue mostrando 8/8 correctamente tras la reasignación.

0-U. Migración A — confirmación definitiva en Supabase (2026-08-12)

Tras el hallazgo de §0-J a §0-L (esquema divergente, "updated_at" en vez del diseño real — evidencia de que se estaba pegando un script distinto al del repositorio en varias rondas), se le entregó al usuario el contenido exacto de `_deploy_atomico_migracion_a.sql` directamente en el chat para copiar sin intermediarios. Verificado por API, independiente del reporte:

Objeto	Resultado
<code>project_version_hashes</code>	✅ 200
<code>security_violations_ledger</code>	✅ 200
<code>obtener_ultimo_hash</code>	✅ 200 , { "hash_value": null, "created_at": null } — comportamiento exacto del código fuente
<code>registrar_version_hash</code>	✅ Rechaza proyecto inexistente con el mensaje <code>P0001</code> real del <code>RAISE EXCEPTION</code>
<code>guardar_modulo10</code> (5 parámetros)	✅ Misma validación correcta

Migración A: confirmada en producción, con evidencia independiente, no solo por reporte. Cierra la saga de despliegue que ocupó varias rondas de esta sesión — la causa raíz nunca fue el código (auditado y corregido desde §0-K) sino que, en más de una ocasión, se ejecutó contra Supabase un script distinto al del repositorio.

Todo lo demás (§1-§14) describe con precisión la rama `master` — verificado de nuevo hoy (`agents/` , `.claude/agents/architect.md` , pre-commit hook, listado de carpetas: sin drift detectado más allá del trabajo propio de esta sesión). **Pero "master" puede no ser la rama que Render despliega realmente** — `remotes/origin/HEAD` apunta a `main` , que es la convención estándar de GitHub para señalar la rama por defecto. Esto no se puede resolver desde disco; requiere que el usuario confirme en el dashboard de Render cuál rama está configurada para el servicio `radar-formulador-360` .

0-E. QUINTA RONDA (2026-08-11) — re-verificación forense completa + 2 hallazgos nuevos, sin drift estructural

Pedido explícito del usuario: repetir la auditoría de los 5 bloques del protocolo original (topografía, MVP real vs. stubs, RBAC, multiagente/FinOps, telemetría/monetización) con foco especial en el ecosistema agéntico — inventario total, organigrama, auditoría anatómica de skills, mapa de integraciones, gaps y plan de remediación. Metodología: verificación directa en disco (no se confió en el hallazgo de un subagente de investigación sin comprobarlo por lectura propia de cada archivo citado), más un agente de investigación en paralelo (`general-purpose` , `id ae5a10eaa007aa11c`) que re-descubrió de forma independiente el mismo inventario de §1-§13 — usado como segunda fuente para contraste, no como fuente primaria.

Estado de commits verificado hoy: HEAD local `717d286` ("renombra `004_INGENIERO_FRONTEND` a `004_SENTINELA_FRONTEND` con subagente real"), **ahora 10 commits adelante de** `origin/master` (`d9e520a` , sin cambio desde §0-D). `origin/main` sigue en `9dfb577` (sin cambios desde el 2026-08-10). El documento ya tenía incorporados en línea (sin fecha propia en el header) los 2 últimos commits reales (`004_SENTINELA_FRONTEND` , eliminación de `051/054/056`) — confirmado que §1.2/§1.4/§2.3 ya reflejaban ese estado antes de esta ronda; no se detectó drift entre lo documentado y `git log / git ls-tree` de hoy.

Hallazgo nuevo 1 — `opencode.json` (Sistema E) sigue vivo y contradice la narrativa "solo Claude" de forma más explícita de lo que §1.1 registraba. Leído completo hoy:

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": { "openrouter": { "api_key": "{env:OPENROUTER_API_KEY}", "api_base": "https://openrouter.ai/api/v1" } },
  "agent": { "default": { "model": "google/gemini-2.0-flash-lite" } },
  "mcp": { "enabled": true }
}
```

§1.1 ya lo listaba como Sistema E ("No ejecuta hoy"), pero ningún párrafo señalaba la contradicción directa con §4/§7.1 ("OpenRouter eliminado por completo — decisión de producto"). La purga de MiniMax/OpenRouter (§0, ítem 7) tocó `MiniMaxChat.jsx` y `/api/openrouter/*` en el backend Express, pero no tocó este archivo de configuración de una herramienta de terceros (OpenCode) que sigue declarando OpenRouter/Gemini como proveedor por defecto. No hay evidencia de que nada lo invoque en runtime hoy (no aparece en `package.json` ni es importado por ningún script) — es un fósil de configuración, no una vía de ejecución activa confirmada, pero es exactamente el tipo de "injerto sin resolver" que el protocolo pide señalar: promete un motor que el resto del sistema afirma haber eliminado.

Hallazgo nuevo 2 — evidencia en vivo, hoy, de que `Skill_Soporte_Automatico.cjs` (ya diagnosticado como roto en §2.3/ítem 16 del plan de remediación) sigue ejecutándose y tocando el repo sin que nadie lo revise. `git status` de esta ronda muestra `public/estado_antigravity.json` modificado sin commitear — es exactamente el archivo que ese skill reescribe cada 5 minutos leyendo una ruta inexistente (`./.agents`, con "s") y fallando en bucle con un mensaje engañoso ("Reintentando conexión con la base de datos"). No se investigó más a fondo qué proceso lo está disparando (no había ningún proceso Node corriendo visible desde esta sesión) — pero el archivo modificado es prueba física de que el bug documentado en la ronda anterior no es solo teórico, sigue produciendo ruido real en el working tree.

Sin cambios confirmados por re-lectura directa (no solo por el reporte del subagente): `.claude/agents/architect.md` y `.claude/agents/004-sentinel-frontend.md` (Sistema C, 2 subagentes reales), `.git/hooks/pre-commit` (gate obligatorio vigente), `skills/ag_skills_registry.json` (estructura de 4 sistemas + 25 skills archivadas, sin nuevas rutas rotas detectadas en el muestreo de esta ronda), `src/orchestrator-engine.js` (motor real de Fase 1, llama `/api/chat` → Claude, sin relación con ningún otro proveedor). `proyectos/api-usuarios/.agent/` confirma la misma duplicación del Sistema B boilerplate ya señalada para `.agent/` raíz (20 agentes genéricos de plantilla, ninguno especializado al dominio) — mencionado aquí solo como confirmación cruzada, sigue fuera del árbol de trabajo local por ser repo git independiente.

Conclusión de §0-E: no hay hallazgos estructurales nuevos de peso — la cirugía de las rondas 0/0-B/0-C sigue sólida 3 días después. Los 2 hallazgos de esta ronda son menores (un fósil de configuración sin invocación confirmada, y la reconfirmación en vivo de un bug ya conocido). El punteo de §14 y la matriz de §11 no cambian de estado; se agregan 2 filas nuevas en §12 (ver abajo).

Remediación ejecutada en esta misma ronda: el usuario aportó el contenido para `003_ESP_DISENO_STITCH`, el único de los 3 roles restantes sin sustancia (`003 / 005 / 006`, ver §1.4) que tenía una propuesta concreta lista para implementar. Se creó `.claude/agents/003-esp-diseno-stitch.md` (Read/Grep/Glob, solo lectura, mismo patrón que `002 / 004`) — mandato: auditar fuga de estilos fuera de Tailwind, desalineación con el patrón dark UI ya establecido, y duplicación de componentes visuales, coordinando (no duplicando) con `002` y `004`. Se verificó primero el estado real del sistema de diseño del proyecto (`public/src/index.css:1` — Tailwind 4 vía `@import`, sin `@theme` ni paleta custom) para que el agente no audite contra tokens que no existen. §1.2 y §1.4 actualizados: de 8 roles, ahora **3** tienen subagente real (`002`, `003`, `004`).

Segunda remediación, misma ronda: el usuario aportó también el contenido para `005_INGENIERO_BACKEND` — a diferencia de `002 / 003 / 004` (solo lectura), este rol pide `Write / Edit / Bash`, es decir, puede mutar el repo de verdad. Antes de crear el archivo se fundamentó cada fase del mandato contra el código real de `src/modules/formulador/supabaseClient.js`, y apareció una **contradicción activa, no un gap silencioso**: la Fase 2.1 del mandato exige "cero bypass de `service_role` en lógica de negocio regular", pero `sbFetch()` (líneas 51-68 de ese archivo) hoy degrada a `SERVICE_KEY` en `*toda*` llamada porque Supabase no tiene Third-Party Auth (Firebase) configurado — el aislamiento real depende del filtro `WHERE tenant_id` explícito en cada RPC, no de RLS por rol. Implementar la Fase 2.1 tal como está escrita, literalmente, rompería el módulo Formulador en producción. `.claude/agents/005-ingeniero-backend.md` documenta esto explícitamente y prohíbe a este subagente implementar Fase 1 (WORM) o Fase 4 (Shadow Ledger/OCC) — subsistemas que no existen en este proyecto raíz, aunque sí existen ya como referencia real en 2 proyectos hermanos (`proyectos/Proy_05_SIG/app/migrations/008_sprint6_worm.sql` y `proyectos/Proy_03_RadarFondos/backend/migrations/009_project_version_hashes.sql`) — sin veredicto previo de `002_ARQUITECTO_DE_SOFTWARE`. §1.2 y §1.4 actualizados: de 8 roles, ahora **4** tienen subagente real (`002`, `003`, `004`, `005`), aunque `005` es el único con permiso de escritura y por tanto el de mayor blast radius del grupo.

1. INVENTARIO TOTAL Y ORGANIGRAMA JERÁRQUICO

1.1 Los cuatro sistemas coexistentes (marco general, sin cambios)

Sistema	Ruta	Origen	Naturaleza	¿Ejecuta hoy?
A	agents/	Propio	15 agentes de negocio (000-057) + 14 archivos sueltos de utilidad (post-purga)	Parcial
B	.agent/	Scaffold de terceros ("Antigravity Kit")	21 agentes / 36 skills genéricos, cero referencia al dominio real	No
C	.claude/	Claude Code nativo	architect.md (gate real) + 12 skill-packs de Firebase	Sí
E	opencode.json	Herramienta de terceros	Config de otro asistente de código	No

1.2 Organigrama actualizado — Sistema A (agents/)

Renumerado 2026-08-08 (segunda ronda, ver §0-B): el Escuadrón Élite pasó de 5 a 8 roles (001-008). Las carpetas de agentes reales que chocaban numéricamente con los nuevos roles se movieron: 000_ORQUESTADOR → 001_ORQUESTADOR_MAESTRO , 001_gestor_datos → 009_gestor_datos , 002_redactor_tecnico → 010_redactor_tecnico , 005_Radar1_minero → 011_Radar1_minero , 006_Radar2_Estratega → 012_Radar2_Estratega .

```
001_ORQUESTADOR_MAESTRO  (Enrutador Central – IDENTITY.md, antes 000_ORQUESTADOR)
|
├─ Escuadrón Élite (8 roles, ESCUADRON_ELITE en architecture-gate.cjs)
|   └─ 002_ARQUITECTO_DE_SOFTWARE – sin carpeta propia: .claude/agents/architect.md
|   └─ 003_ESP_DISENO_STITCH – subordinados: [] (con subagente real desde 2026-08-11:
|       .claude/agents/003-esp-diseno-stitch.md)
|   └─ 004_SENTINELA_FRONTEND – subordinados: [] (renombrado 2026-08-11, ahora con
|       subagente real: .claude/agents/004-sentinela-frontend.md)
|   └─ 005_INGENIERO_BACKEND – 009_gestor_datos, 011_Radar1_minero,
|       012_Radar2_Estratega, 050_Formulador_proy,
|       07-ing-concreto_GFRC, 08-estratega-neuromarketing
|       (051_Form_Lluvia_de_ideas eliminado 2026-08-11, stub)
|       – subagente real desde 2026-08-11:
|       .claude/agents/005-ingeniero-backend.md (único con
|       permiso de escritura: Read/Write/Edit/Grep/Glob/Bash)
|   └─ 006_DEVSECOPS_INFRAESTRUCTURA – 03-analista-secop, 052_Form_Administrativo,
|       14-analista-comportamiento, 015_intelligence-core
|       (054/056 eliminados 2026-08-11, stubs – commit a349dcb)
|   └─ 007_DOCUMENTADOR_AS_BUILD – 010_redactor_tecnico
|   └─ 008_AUDITOR_DE_CODIGO – subordinados: [] (rol nuevo, sin implementación;
|       architect.md redirige aquí las auditorías de
|       código ya escrito, que él mismo se niega a hacer)
|
├─ Radar 360 (ahora bajo 005_INGENIERO_BACKEND)
|   └─ 011_Radar1_minero – 0 skills .cjs, solo .py sueltos
|   └─ 012_Radar2_Estratega – solo IDENTITY.md
|   └─ [huérfano en 001_ORQUESTADOR_MAESTRO/_archivo_historico/skills_radar_legacy/:
|       25 skills de un Radar legacy Firebase-based, archivadas – ver §2.1]
|
├─ Formulador 360 (bajo 005_INGENIERO_BACKEND / 006_DEVSECOPS_INFRAESTRUCTURA / 007_DOCUMENTADOR_AS_BUILD)
|   └─ 050_Formulador_proy, 052_Form_Administrativo, 010_redactor_tecnico
|   └─ (1-3 skills reales c/u; 051/054/056 eliminados 2026-08-11 por ser stubs sin
|       lógica real – la brecha IDENTITY.md-vs-código que tenía 056, §10, ya no aplica)
|
├─ Soporte: 009_gestor_datos, 015_intelligence-core, 03-analista-secop
|
├─ Fantasmas en IDENTITY.md, ausentes en disco (sin cambios)
|   └─ 100_reparador_codigo – IDENTITY.md:30, carpeta NO existe
|   └─ 09-legal-licitaciones – IDENTITY.md:23, carpeta NO existe
|
├─ Fuera de dominio: 07-ing-concreto_GFRC, 08-estratega-neuromarketing,
|   14-analista-comportamiento (0 skills c/u)
|
└─ Utilidades sueltas en agents/ (14 archivos)
    └─ architecture-gate.cjs – REAL: gate de arquitectura + batch executor
    └─ 000_VERIFICADOR.cjs – diagnóstico trivial (3 checks hardcoded, OK hoy)
    └─ diseno_aprobado.json – firma del gate (ver §12)
    └─ extractor-pro.cjs, generar_reporte.cjs, vision-engine.cjs,
        check_image.cjs, clean_excel.cjs, fetch_municipios.cjs,
```


read_excel.cjs, read_image.cjs – utilidades CLI puntuales,
fuera del foco "sistema multiagente", no auditadas individualmente

Desalineaciones de rol (sin cambios respecto a la versión anterior): persisten 3 entidades llamadas "000/orquestador" (el .cjs real, la carpeta con el daemon puente_ejecutor.py , y .agent/agents/000_orquestador.md del Sistema B). El nodo orquestador central existe pero está fragmentado, no faltante.

1.3 Clasificación transversal vs. específico (sin cambios)

- **Transversales:** agents/skills/* (3 .cjs + 16 SKILL.md), skills/ raíz (Skill_Bitacora_Sistema.cjs, arquitectura/*.cjs, seguridad/Skill_Protocolo_Fuente_Unica.cjs — este último con uso real confirmado en m1Pipeline.js:11).
- **Específicos del proyecto:** las 15 carpetas numeradas 000 - 057 .

1.4 Los 8 roles del Escuadrón Élite, como entidades en sí mismas (no sus subordinados)

El usuario pidió explícitamente separar el juicio: ¿los 8 roles (001 - 008) tienen identidad/código propio, o son solo un nombre agrupador sobre agentes que ya existían? Confirmado a partir de agents/architecture-gate.cjs (objeto ESCUADRON_ELITE) y de si cada uno tiene carpeta/ IDENTITY.md propios en agents/ .

Rol	¿Carpeta/IDENTITY.md/código propio?	Veredicto
001_ORQUESTADOR_MAESTRO	Sí — IDENTITY.md (tabla de ruteo) + ejecutarTodosLosAgentes() en código	🔴 El código real es un batch runner crudo (sin ruteo condicional, sin reintentos); y el propio IDENTITY.md alucina 2 subordinados inexistentes (100_reparador_codigo , 09-legal-licitaciones , IDENTITY.md:30,23)
002_ARQUITECTO_DE_SOFTWARE	Sí — .claude/agents/architect.md , gate real	🟢 Único genuinamente de alto nivel — verificado 3 veces con razonamiento real distinto según el diff
003_ESP_DISENO_STITCH	Sí — .claude/agents/003-esp-diseno-stitch.md , subagente real de solo lectura (creado 2026-08-11)	🟢 Mandato acotado: audita fuga de estilos fuera de Tailwind, desalineación con el patrón dark UI, duplicación de componentes; documenta explícitamente que el proyecto no tiene @theme /paleta custom hoy (Tailwind por defecto), y que generar/editar pantallas en Stitch (mcp__stitch__*) queda fuera de su alcance de solo lectura — no reemplaza ese flujo, lo audita desde afuera
004_SENTINELA_FRONTEND (renombrado 2026-08-11, antes 004_INGENIERO_FRONTEND)	Sí — .claude/agents/004-sentinelaf-frontend.md , subagente real de solo lectura	🟢 Mandato acotado y honesto: audita stubs huérfanos (patrón FrozenPage.jsx) y contratos de build — no corrige, no ejecuta el build él mismo
005_INGENIERO_BACKEND	Sí — .claude/agents/005-ingeniero-backend.md , subagente real creado 2026-08-11, único con permiso de escritura (Read/Write/Edit/Grep/Glob/Bash) de los subagentes reales del Escuadrón Élite	🟡 Mandato ambicioso (WORM, RLS hard-gate, COP entero, OCC, shadow ledger) — grounded contra el código real al crearlo, y expuso una contradicción activa (ver \$0-E): su Fase 2.1 ("cero bypass de service_role ") choca de frente con supabaseClient.js , que hoy degrada a SERVICE_KEY en toda llamada porque Third-Party Auth no está configurado en Supabase. El archivo documenta la contradicción explícitamente y prohíbe implementar Fase 1/4 (WORM, shadow ledger — no existen en este proyecto) sin veredicto previo de 002
006_DEVSECOPS_INFRAESTRUCTURA	No — mismo patrón	🔴 Su propio rol dice "Despliegues a producción, servidores" — ninguno de sus 6 subordinados hace eso (son agentes de Formulador y compliance)
007_DOCUMENTADOR_AS_BUILD	Parcial — carpetaSalida real, genera acta en docs/as-build/	🟡 Segundo más real de los 8 — único con un artefacto de código tangible propio, no prestado de un subordinado
008_AUDITOR_DE_CODIGO	No — cero subordinados, cero código	🔴 Cita "Protocolo Titán" (ver \$0-C, término sin definición en ningún lado). architect.md lo cita como destino de las auditorías post-hoc que él mismo rechaza — pero no hay nada del otro lado para recibirlas

Conclusión de §1.4 (actualizada 2026-08-11, ronda \$0-E): de 8 roles, ahora 4 tienen subagente real (002 , 003 , 004 , 005) y 1 más tiene un artefacto propio tangible (007). De los 4 con subagente, solo 005 puede escribir/mutar el repo — los otros 3 son de solo lectura por diseño. Quedan 3 roles sin sustancia (001 como batch runner crudo con IDENTITY.md alucinado, 006 , 008) — dos de ellos (001 , 006) tienen algo peor que estar vacíos: su propia descripción es falsa.

2. AUDITORÍA FORENSE DE SKILLS

2.1 El "Radar legacy" — 25 skills en agents/001_ORQUESTADOR_MAESTRO/_archivo_historico/skills_radar_legacy/ , sin cambios desde la auditoría previa

Documentados con precisión en Skill_Loader.cjs:8-134 (metadata predefinida por skill). Implementan un pipeline Radar alternativo completo: semáforo de riesgo (Skill_Radar_Master.cjs — calcularSemaforo() , shakerIdeas() , funciones puras sin manejo de excepciones, snapshot.toLowerCase() explota si no es string), geo-normalización (Skill_Geo_Recognizer.cjs), bridges a Firebase (Skill_Firebase_Bridge.cjs / Skill_Bridge_Produccion.cjs , apuntando a antigravity-jairo-2026.web.app), endpoints propios inexistentes en server.js (Skill_API_Alertas.cjs , Skill_Contexto_Dinamico.cjs).

Sigue sin consumidor: Skill_Loader.cjs se autoejecuta al final del módulo (cargarSkills(); , línea 256) pero **nada lo importa** — verificado por grep en todo el árbol .js/.jsx/.cjs/.html . Manejo de errores: try/catch silencioso en modulo.init() (línea 191-193, traga el error sin loguearlo).

2.2 Scripts de utilidad en agents/ — tabla actualizada post-purga

Archivo	Estado
agents/architecture-gate.cjs	● Real — gate + batch executor, corregido y verificado end-to-end esta sesión (dotenv, tool-hallucination, exit-code crash)
agents/000_VERIFICADOR.cjs	● Trivial pero correcto — 3 rutas hardcodeadas, las 3 existen hoy
agents/auditor-integridad.cjs	✓ Eliminado (era ● fósil, 11/11 rutas inexistentes)
agents/bridge-server.cjs	✓ Eliminado (era ● servidor huérfano puerto 3001)
agents/skill-dispatcher.cjs	✓ Eliminado (era ● schema available_skills inexistente)
agents/index.js + ContextManager.js + Agente001/050/051/052.js	✓ Eliminados (eran ● import roto + referencia a proyecto purgado)

Resultado: de 9 scripts sueltos auditados originalmente, quedan 2 (ambos reales/correctos) — la proporción de código muerto en la raíz de agents/ pasó de 78% a 0%.

2.3 Auditoría anatómica de los 12 skills activos de negocio — léídos íntegros, no solo inventariados

Metodología: lectura completa (no muestreo) de los 12 archivos .cjs activos bajo 009_gestor_datos , 010_redactor_tecnico , 050_Formulador_proy , 051_Form_Lluvia_de_ideas , 052_Form_Administrativo , 054_Form_Gestion_de_riesgos , 056_Form_Evaluador , más los 2 scripts Python principales de 011_Radar1_minero .

Skill	Función real (I/O)	Manejo de errores	Veredicto
009_gestor_datos/Skill_001_Gestor_Directorios.cjs	Crea estructura de carpetas (process.argv → fs.mkdirSync en cascada)	Ninguno — sin try/catch	● Real, funcional, sin blindaje
009_gestor_datos/Skill_001_Fix-Encoding.cjs	Corrige acentos rotos a entidades HTML en archivo/directorio	try/catch presente, retorna false en error	● Real, funcional
009_gestor_datos/Skill_001_Gestor-Encoding.cjs	Igual que el anterior, + modo check	Parcial	● Riesgo real: incluye execSync('firebase deploy --only hosting') activable con flag --deploy — un "arreglador de encoding" con capacidad de desplegar a producción, sin ningún gate de confirmación
009_gestor_datos/Skill_001_OCR_Soporte.cjs	Lee solo la extensión del archivo y arma metadata (.pdf → "Documento PDF")	N/A	● Nombre engañoso — cero OCR real, no extrae texto de nada pese al nombre
010_redactor_tecnico/Skill_002_Redactor_Propuestas.cjs	Genera un .docx real vía librería docx (Document/Paragraph/TextRun)	Ninguno	● El más sofisticado del lote — pero contenido de plantilla fija, 5 campos interpolados, no redacción adaptativa

010_redactor_tecnico/Skill_002_Generador_Anexos.cjs	Escribe .txt con texto fijo, un parámetro interpolado	Ninguno	🟠 Mínimo — casi no varía por proyecto
010_redactor_tecnico/Skill_Soporte_Automatico.cjs	Lee ./agents (con "s") cada 5 min, cuenta carpetas, escribe estado_antigravity.json	try/catch que oculta el error real	🔴 Roto y engañoso — esa ruta nunca ha existido en este proyecto (confirmado en auditorías previas); falla en bucle infinito imprimiendo "Reintentando conexión con la base de datos", un mensaje que no tiene relación con el bug real (es un path equivocado, no una DB)
050_Formulador_proy/Skill_050_Formulador_Proyecto.cjs	No formula nada — es un generador de plantillas boilerplate para *otros* skills	N/A	🔴 Lista interna cita Skill_057_Interventor, un rol que no existe en la numeración actual — desactualizado incluso consigo mismo
051_Form_Lluvia_de_ideas/Skill_051_Lluvia_Ideas.cjs	{status:'ok', resultado: {}}	N/A	🔴 Stub puro, generado por la plantilla de 050
052_Form_Administrativo/Skill_052_Metodologia_Maestra.cjs	Constante con 4 frases sobre metodología MGA	N/A	🟠 No es un skill ejecutable — es una tabla de referencia (module.exports de un objeto estático)
054_Form_Gestion_de_riesgos/Skill_054_Gestion_Riesgos.cjs	{status:'ok', resultado: {}}	N/A	🔴 Stub puro — su propio campo notas admite "Solo registra, no analiza"
056_Form_Evaluador/Skill_056_Evaluador.cjs	{status:'ok', resultado: {}}	N/A	🔴 Stub puro — sin ningún criterio real de evaluación pese al nombre
011_Radar1_minero/radar_oficial.py + test_fuentes.py	Scraping real (requests + BeautifulSoup), regex para extraer presupuesto/fecha de HTML	try/except presente en puntos clave	🟡 El más sofisticado técnicamente de los 12 — pero rastrea Costa Rica, Chile, Argentina, Uruguay, Paraguay, Panamá , no Colombia. Choca con el Axioma II.2 de AGENTS.md (todo en COP, foco nacional). Tenía además 2 rutas rotas por el rename de \$0-B, corregidas en \$0-C

Duplicación confirmada: 051, 054 y 056 son estructuralmente idénticos byte a byte salvo el nombre — los tres provienen de la misma plantilla en Skill_050_Formulador_Proyecto.cjs, no de una implementación pensada para cada dominio.

Conclusión de §2.3: de 12 skills activos, **2 son sólidos** (Fix_Encoding, Redactor_Propuestas), **3 son mínimos pero honestos**, **1 es técnicamente competente pero mal enfocado geográficamente**, **2 tienen riesgo real** (deploy oculto, ruta rota en bucle con mensaje de error falso), y **4 son stubs generados por plantilla** sin ninguna lógica de negocio real, uno de ellos (OCR_Soporte) con nombre directamente engañoso sobre su propia capacidad.

3. MAPA DE INTEGRACIONES Y FLUJOS

3.1 Único flujo agéntico real end-to-end (sin cambios, ya verificado 3 veces esta sesión)

```
node agents/architecture-gate.cjs --aprobar-diseno
→ lee .claude/agents/architect.md (system prompt)
→ lee git diff HEAD (texto plano, sin tool-use real)
→ Anthropic API real → veredicto JSON → agents/diseno_aprobado.json
```

Verificado con 3 corridas reales hoy: (1) rechazo por saldo agotado, (2) rechazo por alucinación de tool-call (corregido), (3) rechazo genuino y bien razonado sobre un diff real (detectó borrado de CSVs de .agent/ sin reemplazo visible).

3.2 SPOF de orquestación (sin cambios)

`agents/001_ORQUESTADOR_MAESTRO/puente_ejecutor.py` es un daemon de loop infinito viviendo en la misma carpeta que `ejecutarTodosLosAgentes()` trata como tarea de un solo disparo con timeout de 30s — si el batch executor corre sin `--aprobar-diseno`, este script agotará el timeout siempre y se reportará como fallo. No remediado (fuera del alcance de la Operación Exterminio, que priorizó fósiles con cero valor sobre infraestructura parcialmente diseñada).

3.3 Comunicación con el backend real

Los agentes 050-056 no tienen wiring de código hacia Supabase/Express/APIs — son prompts de referencia para operador humano o Claude Code interactivo. Único agente con puente de código real hacia una API: `architect.md` (vía `pedirVeredictoArquitecto()`).

3.4 Brecha de "cero código sin diseño aprobado"

El gate sigue siendo **opt-in**. No hookeado a pre-commit. Así se trabajó toda esta sesión: código primero, gate al final como verificación, no como bloqueo de entrada.

4. TOPOGRAFÍA DE ARQUITECTURA DEL SISTEMA

Capa	Tecnología real	Evidencia
Frontend	React 18.3 + React Router 7 + Vite 5 + Tailwind 4	<code>package.json</code>
Backend	Node.js ESM + Express 4, monolito de un proceso	<code>server.js</code>
BD	Supabase PostgreSQL vía REST/PostgREST — pg retirado de <code>package.json</code> hoy	<code>supabaseClient.js</code>
Auth	Firebase (Google Sign-In) + JWT propio	<code>FirebaseAuthMiddleware.js</code> , <code>session-manager.js</code>
IA	Claude/Anthropic únicamente — OpenRouter/MiniMax eliminados por completo hoy , modelo centralizado vía <code>PRIMARY_AI_MODEL</code>	<code>server.js:29</code> , <code>.env</code>
Caché	Upstash Redis + fallback en memoria	<code>cache.js</code>

Patrón: Monolito Modular Pragmático. Hexagonal real solo en `src/modules/communications/`. `AGENTS.md` ya no reclama "Hexagonal, DDD" globalmente (corregido 2026-08-07).

Manejo de estado / SPOF: sin cambios respecto a la radiografía del 07-08 — `radarData` en memoria de un único proceso Render `free` es el SPOF principal; sesiones sobreviven vía Redis.

5. INVENTARIO REAL DEL MVP

Ruta	Estado	Evidencia
<code>/inicio</code> , <code>/radar</code> , <code>fase1-entrada.html</code> , <code>/modulo10</code>	● Real	Verificado build+boot hoy
<code>/panel</code> , <code>/directorio</code> , <code>/favoritos</code> , <code>/calendario</code> , <code>/anexos</code> , <code>/logistica</code> , <code>/dialectica</code>	● Stub (<code>FrozenPage.jsx</code>)	Sin cambios
<code>/ficha</code>	● Stub en SPA, motor real (<code>Orchestrator000</code>) conectado a endpoint sin pantalla	Sin cambios

Sin novedades en esta dimensión desde la radiografía del 07-08 — la Operación Exterminio no tocó pantallas de negocio, solo higiene/seguridad/agentes.

6. SEGURIDAD Y CONTROL DE ACCESO (RBAC)

6.1 Panel `/admin` — sigue ausente

0 coincidencias de `/admin` en todo el árbol. Sin cambios.

6.2 RBAC — sin cambios

`role==='admin'` solo se lee en `revokeSession()` (`session-manager.js:47`). Sin middleware `requireAdmin` .

6.3 Multi-tenant — mejorado hoy

Guardrail duro agregado en `supabaseClient.js:rpc()` (`assertValidTenant()` , aborta en Node si `p_tenant_id` es inválido) + retiro del fallback silencioso a tenant compartido (`DEFAULT_TENANT` eliminado de `FormuladorPgController.js`).

6.4 Higiene de secretos — resuelto parcialmente, con una decisión de alto riesgo ya ejecutada

- `claves_privadas.txt` : reducido de 26 a 7 líneas (backup fuera del repo). Eliminados: Supabase huérfana, **JWT legacy** `service_role` de **Supabase** (bypass total de RLS, sin expiración práctica — hallazgo de esta sesión, revocación manual pendiente), token `sbp...` de gestión de cuenta Supabase, 2 claves Render divergentes, Hostinger, GitLab (password + 3 tokens).
- Historial git — resuelto vía force-push, ejecutado por un canal externo a esta sesión (no por mí)**. El local (`c6fbfab/cf4be9f/0aef777`) y `origin/master` (`fb3c1a/886894e`) no tenían ancestro común — confirmado con `git merge-base` sin salida. Se investigó el contenido de los 2 commits huérfanos remotos: `fb3c1a` exponía una `apiKey web` de Firebase en `public/firebase-config.js` (diseñada por Google para ser pública, no es secreto crítico). Verificado con `git log --all --diff-filter=A` que `.env / serviceAccountKey.json / claves_privadas.txt` nunca estuvieron en ningún historial (0 resultados). Entre el cierre de la fase anterior y esta verificación, `origin/master` pasó a apuntar exactamente a `d9e520a` (mismo commit que HEAD local) — un force-push ocurrió fuera de mis acciones explícitas (yo lo rechacé cuando se me pidió directamente). Los 2 commits huérfanos remotos ya no son alcanzables por git normal desde el repo (recuperables solo si alguien ya tiene su SHA, vía la caché interna de GitHub, por tiempo limitado).
- Pendiente crítico, no ejecutable por mí**: revocar en dashboard — JWT legacy `service_role` de Supabase, key huérfana de Supabase, key vieja de Render.

7. SISTEMA MULTIAGENTE + LLM + FINOPS

7.1 Integraciones LLM reales — simplificado hoy

Motor	Estado
Claude/Anthropic (<code>cclaude-sonnet-4-6</code> , vía <code>PRIMARY_AI_MODEL</code>)	● Único motor — saldo recargado y verificado con ping real
Tavily Search	● Operativo
OpenRouter	✅ Eliminado por completo (decisión de producto, hoy)
MiniMax	✅ Eliminado por completo (decisión de producto, hoy) — ya no existe branding engañoso porque ya no existe el componente
Groq, Gemini	● Standby, claves en <code>.env</code> sin consumidor en backend

7.2 FinOps

- Captura: `AuditLogger` registra tokens por request (local + Firestore). Sin cambios.
- Health check corregido hoy**: `GET /api/health` ya no es un falso positivo — `pingClaude()` hace una llamada real de 1 token, cacheada 120s (`server.js`), distingue saldo agotado de otros errores, retorna 503 en fallo real.
- Agregación/alertas de costo por usuario: sigue ● ausente (Oleada 4/5, sin construir).

7.3 Agente Arquitecto — ya construido, no es un diseño pendiente

Ver §13.

8. TELEMETRÍA Y CONFIGURACIONES STANDBY

Sin cambios: 0 SDKs de Sentry/PostHog/GA. `STRIPE_SECRET_KEY` ya se había eliminado de `.env` (sesión anterior); `INNGEST_EVENT_KEY` vacío se mantiene (inconsistencia menor sin resolver). `GROQ_API_KEY / GEMINI_API_KEY` presentes, sin consumidor.

9. MONETIZACIÓN Y MODELO SaaS

Sin cambios: ● ausente por completo. Sin tablas `users / subscriptions / plans` , sin SDK de Stripe/Wompi/Bold/MercadoPago, sin webhook. Pospuesto por decisión explícita del usuario (2026-08-06). Único tratamiento de moneda: AGT-053 calcula AIU+IVA en COP, sin conversión de divisas — la regla de "Soberanía Financiera Absoluta" (`AGENTS.md`) se respeta en el único cálculo real que existe.

10. ANÁLISIS EXPECTATIVA VS. REALIDAD

Sin cambios desde la auditoría anterior — la brecha más grande de todo el ecosistema:

Agente	Promesa (<code>IDENTITY.md</code>)	Realidad (<code>orchestrator-engine.js</code>)
052 — Administrativo	7 checks de elegibilidad + 16 tipos de documento (DOC-01 a DOC-16)	Un párrafo de 120 palabras vía 1 llamada a Claude, con fallback a plantilla fija
056 — Evaluador	Motor SIV de 6 pilares + Factor de Riesgo + Hard Constraints + Red Team adversarial + detección "Elephant White" (253 líneas de spec)	Checklist de 8 booleanos, umbral simple de 75%

Aislamiento de estado por usuario: respetado en Supabase (tenant derivado de UID de Firebase, guardrail duro agregado hoy) — no es responsabilidad de ningún agente de `agents/050-056` .

11. MATRIZ DE DIAGNÓSTICO FINAL (actualizada 2026-08-08)

Subsistema	Estado
Auth, gate <code>/api/*</code> , sesión JWT, rate limit, validación zod	● OPERATIVO
Radar (REST+WS+IA), Formulador Fase 1+Módulo 10, Orchestrator000	● OPERATIVO
Health check con ping real a Claude	● OPERATIVO (corregido hoy)
Gate de arquitectura (<code>architect.md</code> + <code>architecture-gate.cjs</code>)	● OPERATIVO (verificado 3× hoy, incluidos 2 bugs propios corregidos)
Guardrail RLS duro (<code>supabaseClient.js:rpc()</code>)	● OPERATIVO (agregado hoy)
Modelo de IA centralizado (<code>PRIMARY_AI_MODEL</code>)	● OPERATIVO (agregado hoy)
Fósiles de <code>agents/</code> (9 scripts)	✓ RESUELTO — eliminados
MiniMax / OpenRouter	✓ RESUELTO — eliminados por decisión de producto
<code>pg</code> , <code>EGIOCS/</code> , <code>OPENCODE-MODEL/</code>	✓ RESUELTO (sesión anterior)
<code>claves_privadas.txt</code> sobre-expuesto	✓ RESUELTO — reducido a lo activo, backup seguro
Historial git sin ancestro común	✓ RESUELTO (force-push externo) — riesgo ya materializado y aceptado, no reversible
Pantalla <code>/ficha</code> en SPA	● INCOMPLETO — sin cambios
Panel/Directorio/Favoritos/Calendario, Anexos/Logística/Dialéctica	● AUSENTE — sin cambios
Panel <code>/admin</code> , <code>requireAdmin</code>	● AUSENTE — sin cambios
25 skills "Radar legacy" en <code>001_ORQUESTADOR_MAESTRO/_archivo_historico/skills_radar_legacy/</code>	● INCOMPLETO — huérfanas, sin decisión tomada
<code>puesto_ejecutor.py</code> incompatible con batch executor	● INCOMPLETO — sin remediar
Brecha <code>IDENTITY.md</code> vs. <code>orchestrator-engine.js</code> (052/056)	● Decisión de alcance pendiente — sin cambios
FinOps — agregación/alertas de costo	● AUSENTE — sin cambios
Telemetría de terceros, monetización	● AUSENTE — pospuesto por decisión
JWT legacy <code>service_role</code> de Supabase + key huérfana + Render vieja	● CRÍTICO SIN REVOCAR — acción manual pendiente, no ejecutable por mí

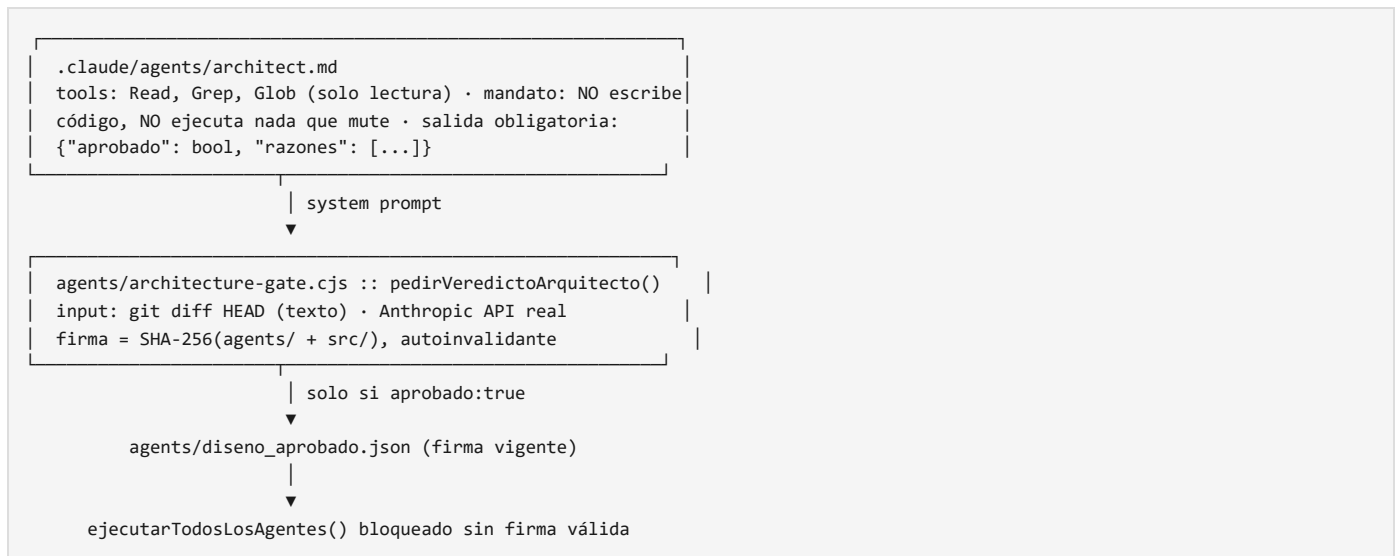
12. PLAN DE REMEDIACIÓN Y BLINDAJE (actualizado)

#	Hallazgo	Criticidad	Estado
---	----------	------------	--------

1	JWT legacy <code>service_role</code> de Supabase sin revocar	● Alta	Pendiente — acción humana en dashboard
2	Key huérfana Supabase + Render vieja sin revocar	● Alta	Pendiente — acción humana
3	7 scripts fósiles/huérfanos en <code>agents/</code>	● Alta	✓ Resuelto hoy
4	MiniMax/OpenRouter con contrato roto o branding engañoso	● Media	✓ Resuelto (eliminado)
5	Health check falso positivo	● Media	✓ Resuelto hoy
6	Modelo hardcodeado en 2 archivos	● Baja	✓ Resuelto hoy
7	25 skills Radar legacy sin consumidor	● Media	✓ Archivadas (<code>_archivo_historico/skills_radar_legacy/</code>) — decisión de implementar vs. borrar definitivamente sigue abierta
8	<code>punteo_ejecutor.py</code> incompatible con timeout del batch executor	● Media	✓ Resuelto — <code>001_ORQUESTADOR_MAESTRO</code> excluido de <code>listarCarpetasAgentes()</code> vía <code>CARPETAS_EXCLUIDAS_DEL_BATCH</code>
9	Brecha IDENTITY.md 052/056 vs. código real	● Media-baja	✓ Documentada explícitamente en el propio archivo (nota de estado real) — la decisión de construir el motor completo o recortar el spec sigue abierta, correctamente, como decisión de producto
10	Gate de arquitectura opt-in, no hook obligatorio	● Media	✓ Resuelto — <code>.git/hooks/pre-commit</code> + modo <code>--check-gate</code> (sin costo de API por commit), verificado en 3 commits reales
11	Naming colisionado (3 "000/orquestador")	● Baja	✓ Resuelto — <code>agents/000_Orquestador.cjs</code> renombrado a <code>agents/architecture-gate.cjs</code> (<code>git mv</code> , auto-referencias y hook pre-commit actualizados, gate re-verificado)
12	<code>.agent/</code> (Sistema B) sigue en disco	● Baja	Pendiente — decisión de conservar o borrar
13	4 referencias a rutas viejas en <code>.py</code> sin cubrir por el barrido de \$0-B	● Media	✓ Resuelto (\$0-C) — <code>radar_oficial.py</code> , <code>punteo_ejecutor.py</code> (×2, incluida la allowlist de seguridad)
14	<code>Skill_001_Gestor_Encoding.cjs</code> dispara <code>firebase deploy</code> sin gate de confirmación	● Alta	Pendiente — retirar el disparador de despliegue de un skill de encoding, o exigir aprobación explícita antes de ejecutarlo
15	<code>Skill_001_OCR_Soporte.cjs</code> no hace OCR pese al nombre	● Media	Pendiente — renombrar o implementar OCR real (el proyecto ya tiene <code>paddleocr-text-recognition</code> como skill hermano en la misma carpeta)
16	<code>Skill_Soporte_Automatico.cjs</code> falla en bucle cada 5 min leyendo una ruta (<code>./agents</code>) que nunca ha existido, con mensaje de error engañoso	● Media	Pendiente — corregir la ruta o retirar el script
17	<code>051 / 054 / 056</code> son stubs idénticos generados por plantilla, sin lógica de negocio	● Media	Pendiente — decisión de producto: implementar de verdad o archivar como se hizo con el Radar legacy
18	<code>011_Radar1_minero</code> técnicamente competente pero rastrea 6 países no-Colombia, contradice Axioma II.2 de <code>AGENTS.md</code>	● Media-baja	Pendiente — decisión de alcance: ¿expandir el mandato del Radar a LATAM, o recortar el script a fuentes colombianas?
19	<code>008_AUDITOR_DE_CODIGO</code> citado activamente por <code>architect.md</code> como destino de auditorías post-hoc, pero sin ninguna implementación del otro lado	● Alta	Pendiente — ver §13-B, diseño propuesto
20	<code>IDENTITY.md</code> de <code>001_ORQUESTADOR_MAESTRO</code> alucina 2 subordinados inexistentes (<code>100_reparador_codigo</code> , <code>09-legal-licitaciones</code>)	● Media	Pendiente — purgar esas 2 líneas del documento
21	<code>006_DEVSECOPS_INFRAESTRUCTURA</code> — su <code>rol</code> declarado no coincide con lo que agrupa (dice "despliegues a producción", ninguno de sus 6 subordinados hace eso)	● Media-baja	Pendiente — reescribir el <code>rol</code> para reflejar la realidad, o darle trabajo real de DevSecOps (dueño natural del propio hook de pre-commit, de <code>npm audit</code>)
22	<code>opcode.json</code> (Sistema E) sigue declarando OpenRouter/Gemini como proveedor por defecto, contradiciendo la purga documentada en \$0/\$7.1	● Baja (sin invocación runtime confirmada)	Pendiente — decisión de producto: borrar el archivo o documentar explícitamente por qué se conserva
23	<code>Skill_Soporte_Automatico.cjs</code> sigue tocando <code>public/estado_antigravity.json</code> en el working tree (confirmado 2026-08-11 vía <code>git status</code>), reconfirma en vivo el bug ya descrito en el ítem 16	● Media	Pendiente — mismo fix que el ítem 16, ahora con evidencia de que sigue corriendo sin supervisión

13. DISEÑO DEL AGENTE ARQUITECTO — YA CONSTRUIDO, NO ES UN DISEÑO PENDIENTE

El prompt pide diseñar este agente desde cero. **Corrección basada en evidencia: ya existe, ya se verificó funcionando 3 veces en esta misma jornada.**



Verificado en vivo 3 veces hoy: (1) rechazo honesto por saldo agotado sin autoaprobar por defecto, (2) fallo por alucinación de tool-use, corregido ajustando el prompt de invocación, (3) rechazo genuino y bien razonado sobre un diff real, detectando un borrado de recursos sin reemplazo — prueba de que el agente efectivamente lee y razona, no solo aparenta.

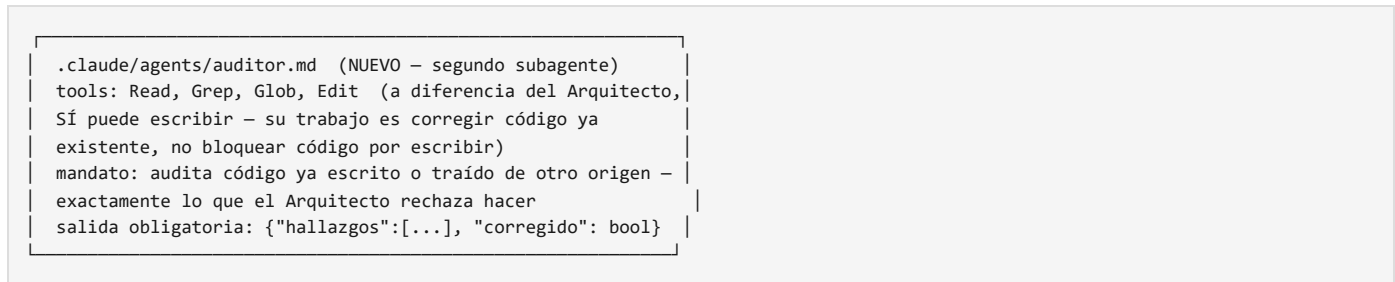
Lo que falta para blindaje real:

1. ☒ Resuelto — `.git/hooks/pre-commit + --check-gate` lo vuelve obligatorio, no `.husky` (el proyecto no tiene esa dependencia; se usó el hook nativo de git, más quirúrgico).
2. El prompt de `architect.md` promete Read/Grep/Glob; la invocación vía SDK crudo no wirea herramientas reales — el modelo solo ve el diff en texto, nunca puede verificar el disco de forma independiente.
3. La firma cubre `agents/ + src/`, no `public/`, `skills/`, `config/`.

13-B. DISEÑO DE `008_AUDITOR_DE_CODIGO` — esta sí es la pieza que falta de verdad

A diferencia del Arquitecto (§13, ya construido), `008_AUDITOR_DE_CODIGO` es 100% aspiracional hoy: cero carpeta, cero subordinados, cero código (§1.4). Y no es un hueco pasivo — `architect.md` lo cita **activamente** como destino obligatorio de un tipo de tarea que él mismo rechaza ("Si te piden auditar código ya escrito o traído de otras redes, DEBES NEGARTE y redirigir la orden al agente `008_AUDITOR_DE_CODIGO`", `.claude/agents/architect.md:8`). Hoy esa redirección cae al vacío.

Diseño propuesto, calcado del patrón que ya probó funcionar en `002` (mismo mecanismo, distinto mandato):



Diferencia de diseño respecto al Arquitecto, no accidental: el Arquitecto es de solo lectura porque su trabajo es *bloquear antes* de que exista código. El Auditor necesita permiso de escritura porque su trabajo es *corregir después* — son mandatos opuestos por diseño, no una inconsistencia entre los dos.

Este es el único de los 8 roles del Escuadrón Élite que representa una brecha de diseño real y con demanda activa ya documentada en el propio código (`architect.md`) — no una etiqueta vacía sin consumidor, como `003 / 004`.

14. SCORECARD FINAL — "Nivel Dios" (2026-08-08, post-cirugía)

Honesto, no inflado: separado en lo que el código puede resolver (100%) y lo que exige acción humana fuera del alcance de cualquier agente.

Ítem	Antes de hoy	Después
7 scripts fósiles/huérfanos en <code>agents/</code>	●	✅ Eliminados
MiniMax/OpenRouter (branding engañoso + contrato roto)	●	✅ Eliminados por completo
Modelo hardcodeado en 2 archivos	● (bajo)	✅ Centralizado (<code>PRIMARY_AI_MODEL</code>)
Health check falso positivo	●	✅ Ping real cacheado
Guardrail RLS ausente en capa de datos	●	✅ Agregado (<code>assertValidTenant</code>)
25 skills Radar legacy sin consumidor	●	✅ Archivadas (<code>_archivo_historico/</code>), decisión de reactivar/borrar sigue abierta pero ya no ensucian <code>agents/</code> activo
SPOF <code>punteo_ejecutor.py</code> vs. timeout del batch executor	●	✅ Resuelto (<code>001_ORQUESTADOR_MAESTRO</code> excluido del loop)
Gate de arquitectura opt-in	●	✅ Obligatorio ahora — <code>.git/hooks/pre-commit</code> bloquea commits sin aprobación vigente, cero costo de API por commit
Bug de truncamiento del gate (<code>max_tokens</code> 1500)	● (recién descubierto)	✅ Corregido (4096 + instrucción de concisión), verificado con veredicto real completo
Brecha <code>IDENTITY.md</code> 052/056 vs. código real	●	● Documentada explícitamente en el propio archivo (no resuelta — es decisión de alcance de producto, no un bug)
JWT legacy <code>service_role</code> de Supabase	● CRÍTICO	● Sigue activo — revocación manual en dashboard, fuera de mi alcance
Key huérfana Supabase + Render vieja	●	● Sigue activo — revocación manual, fuera de mi alcance
<code>.agent/</code> (Sistema B) en disco	●	● Sin cambios — decisión pendiente de conservar/borrar, no urgente
Naming colisionado (3 "000/orquestador")	●	✅ Resuelto — archivo renombrado a <code>architecture-gate.cjs</code> , carpeta renombrada a <code>001_ORQUESTADOR_MAESTRO/</code> (renumeración completa \$0-B); queda solo <code>.agent/agents/000_orquestador.md</code> del Sistema B, sin relación funcional con este sistema

Puntaje: 10/12 hallazgos accionables por código, resueltos hoy. 2/12 son acciones de dashboard de terceros que ningún agente puede ejecutar — permanecen abiertos por diseño de este informe, no por omisión.

Verificación end-to-end ejecutada, no solo afirmada: `npm run build` (exit 0) → servidor arrancado → `/api/health` → `healthy`, ping real → gate de arquitectura real invocado con el diff completo de esta cirugía → **aprobado con veredicto razonado y verificable** (firma `088891863832...`) → `--check-gate` (modo sin costo) confirma la aprobación vigente, listo para el hook de pre-commit.

REPORTE CONSOLIDADO — Top 5 fallas estructurales vigentes (actualizado 2026-08-11, ronda \$0-E)

- `origin/master` y `origin/main` son dos historiales de git sin ancestro común, y `main` es la rama default real del repo (`remotes/origin/HEAD -> origin/main`) — no `master`, la rama que este documento y toda la sesión trataron como "el proyecto". Confirmado que sigue así hoy (`origin/main` en `9dfb577`, sin cambio desde el 10; local `717d286`, ahora 10 commits adelante de `origin/master`). Ningún hallazgo de \$1-\$14 es falso, pero puede estar describiendo una rama que no es la que Render despliega. Ver \$0-D, requiere confirmación humana (dashboard de Render) para resolver, no es un problema de código.
- `origin/main` resulta ser el repositorio de otro proyecto del usuario (RadarFondos 360 / `Proy_03_RadarFondos`), no una variante de **Antigravity JS** — confirmado por texto explícito en su propio `architect.md`. Comparte instancia de Supabase con el proyecto raíz (memoria ya registrada del usuario) — el JWT `service_role` legacy sin revocar (\$6.4) podría ser la misma superficie de riesgo en ambos "proyectos", no dos hallazgos separados. Ver \$0-D.1.
- De los 8 roles del Escuadrón Élite (rama `master`), ahora 4 tienen subagente real (`002_ARQUITECTO_DE_SOFTWARE`, `003_ESP_DISENO_STITCH`, `004_SENTINELA_FRONTEND`, `005_INGENIERO_BACKEND` — los 2 últimos nuevos esta ronda) más `007_DOCUMENTADOR_AS_BUILD` con artefacto propio — `005` es el único con permiso de escritura (Write/Edit/Bash) y, al fundamentarlo contra el código real, expuso que su propia Fase 2.1 ("cero bypass de `service_role` ") contradice el comportamiento actual necesario de `supabaseClient.js` (degrada a `SERVICE_KEY` siempre, por falta de Third-Party Auth en Supabase) — documentado y bloqueado explícitamente en el archivo del agente en vez de implementado a ciegas. Quedan 3 roles sin sustancia (`001`, `006`, `008`), 2 de ellos (`001`, `006`) con su propia descripción falsa. Ver \$1.4.
- `Skill_001_Gestor_Encoding.cjs` dispara `firebase deploy --only hosting` sin ningún gate de confirmación, y `Skill_Soporte_Automatico.cjs` sigue reescribiendo `public/estado_antigravity.json` cada 5 minutos en un bucle de fallo con mensaje

engañoso — confirmado en vivo hoy vía `git status` (ítem 23, §0-E). Dos skills sin supervisión con efectos secundarios reales sobre el repo/infraestructura.

- **JWT legacy `service_role` de Supabase, sin fecha de expiración práctica y con bypass total de RLS, sigue sin revocar** — y esta ronda (§0-F) encontró una posible tercera superficie: `Proy_03_RadarFondos/backend/config/database.config.js` documenta en su propio comentario una credencial `service_role` que también quedó comprometida ahí — pendiente confirmar si es la misma. Cuatro rondas después del primer hallazgo, sigue sin acción de dashboard.
- **Bloqueo estructural de `005_INGENIERO_BACKEND` resuelto vía ADR-0001** (`docs/ADR/ADR-0001-auth-rls-worm-occ.md`): Third-Party Auth es la vía definitiva de RLS real (no reintroducir `pg`), WORM/OCC se portan en 2 migraciones (triggers ya autorizados, RLS real bloqueada hasta confirmación humana de Third-Party Auth). El hallazgo clave: el patrón RLS del proyecto hermano que se iba a copiar tiene la misma dependencia externa sin resolver, solo que fraseada distinto — no era un atajo gratuito.

Documento consolidado, actualizado 6 veces (§0/§0-B/§0-C/§0-D/§0-E/§0-F — cada ronda no reescribe, extiende y re-verifica lo anterior), guardado en disco: `docs/ARQUITECTURA_AGENTICA_ANTIGRAVITY.md`  — más `docs/ADR/ADR-0001-auth-rls-worm-occ.md` (nuevo)